Unit-1 - Introduction

MULTIMODAL MACHINE LEARNING

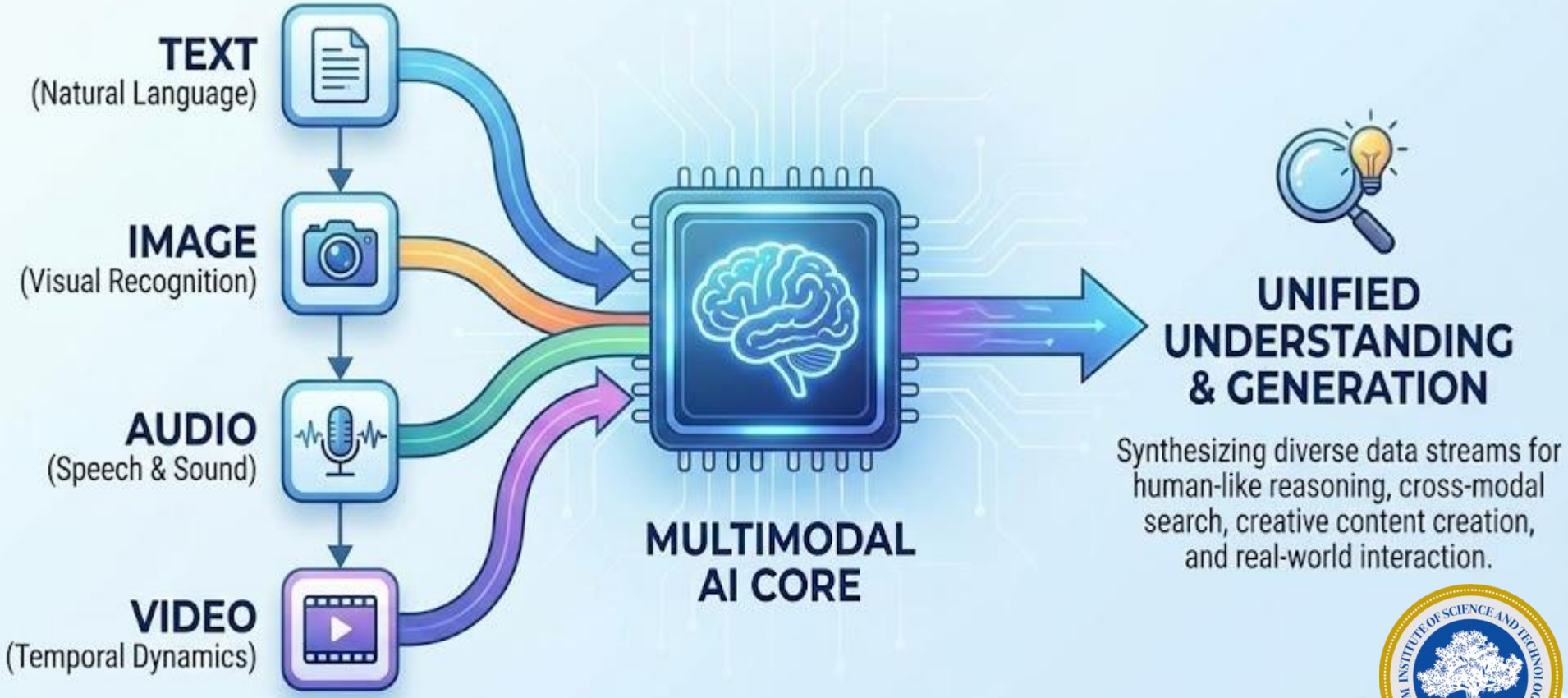
Multimodal AI: Converging Senses for Human-Like Intelligence

- AI systems that can simultaneously process, understand, and generate multiple types of data (modalities) to solve complex problems, mimicking human perception.

The "Senses" (Inputs):

- **Text:** Documents, code, logs.
- **Visual:** Images, video, 3D scans.
- **Audio:** Speech, environmental sounds.
- **Sensor Data:** LiDAR, thermal, IoT signals.

WHAT IS MULTIMODAL AI?



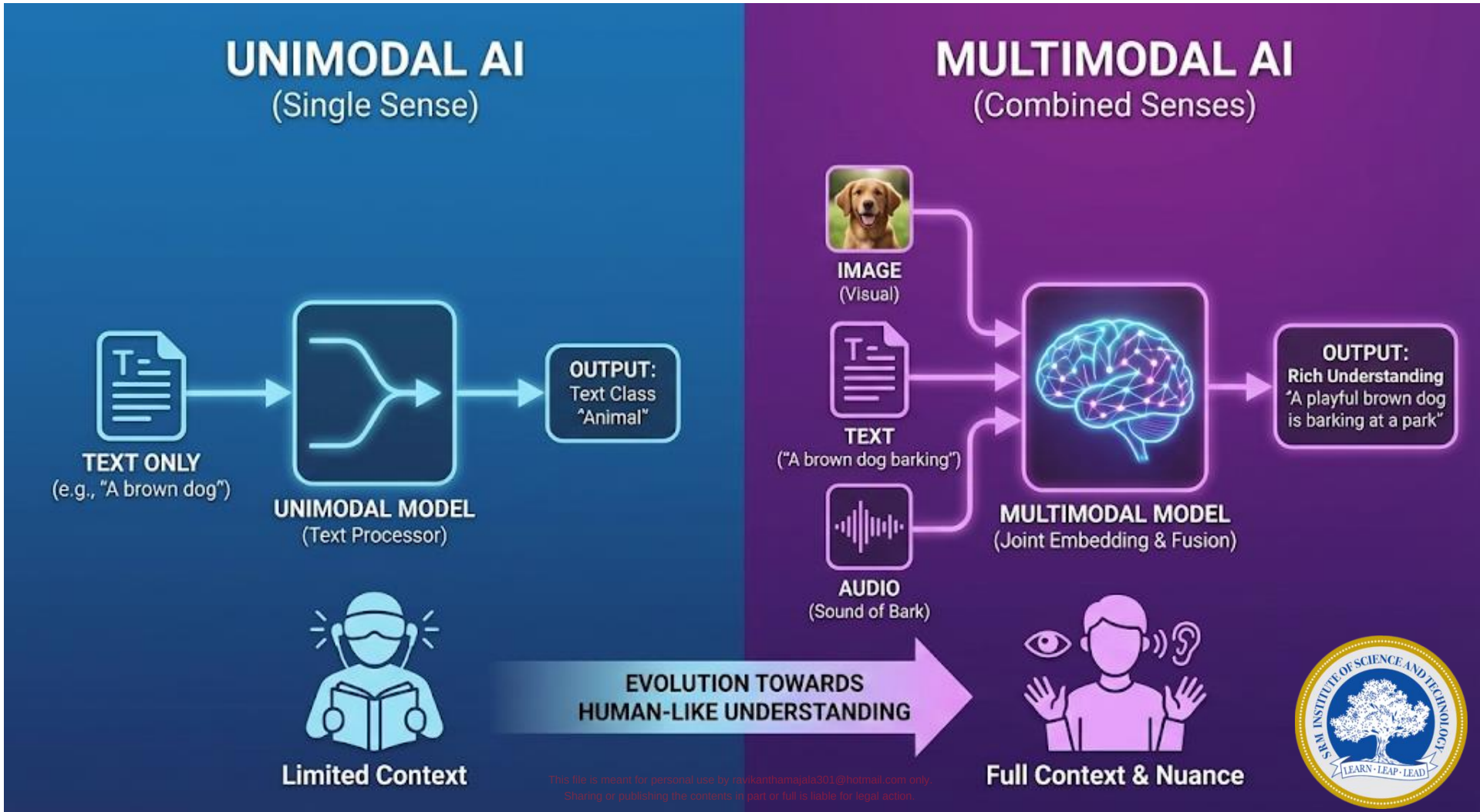
How It Works

- Instead of training separate models for text and image, Multimodal AI uses **Joint Embedding Spaces**. It maps different data types into a shared understanding, allowing the model to see connections between a photo of a cat and the word "cat.“

Key Advantages:

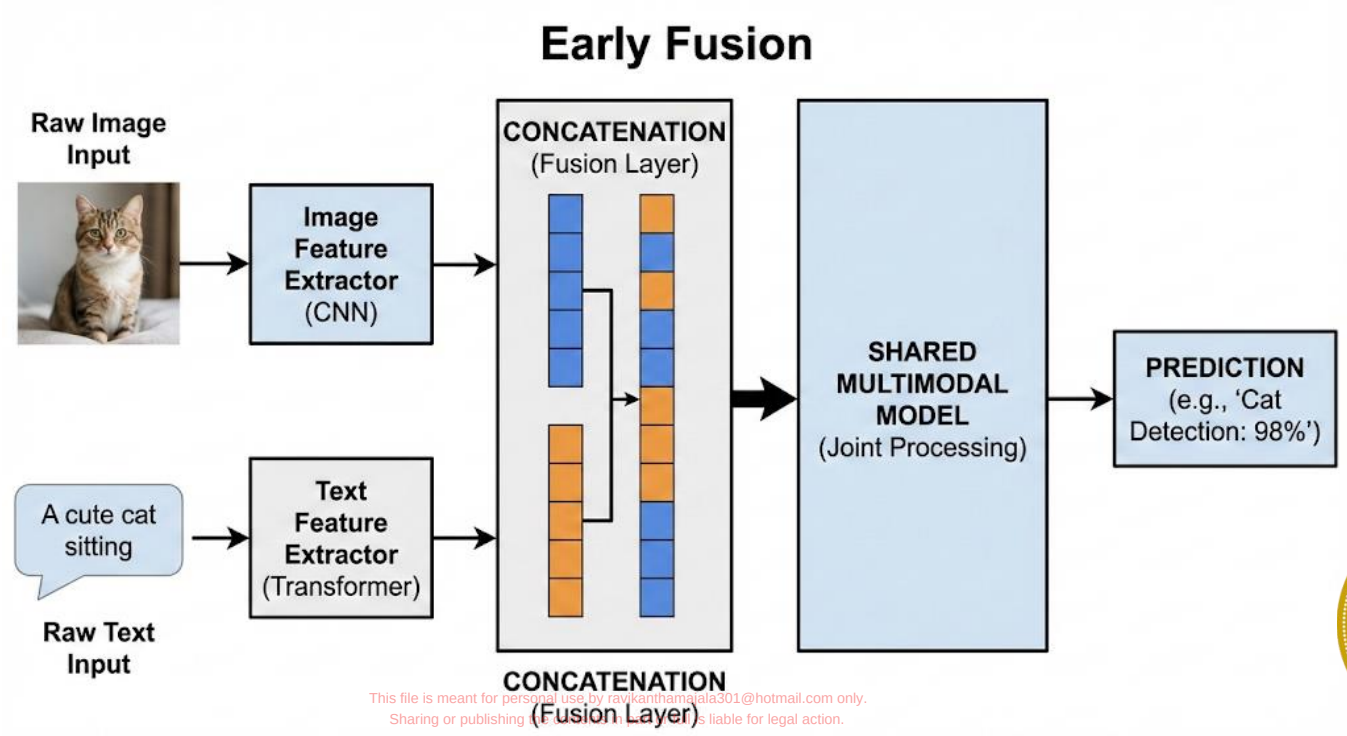
- **Deep Context:** Can analyze a video (visual) and its transcript (text) together for nuance.
- **Ambiguity Resolution:** Visual cues can clarify ambiguous text, and voice tone can clarify intent.
- **Versatility:** One model can write code, analyze a chart, and describe a screenshot.

Unimodal vs Multimodal



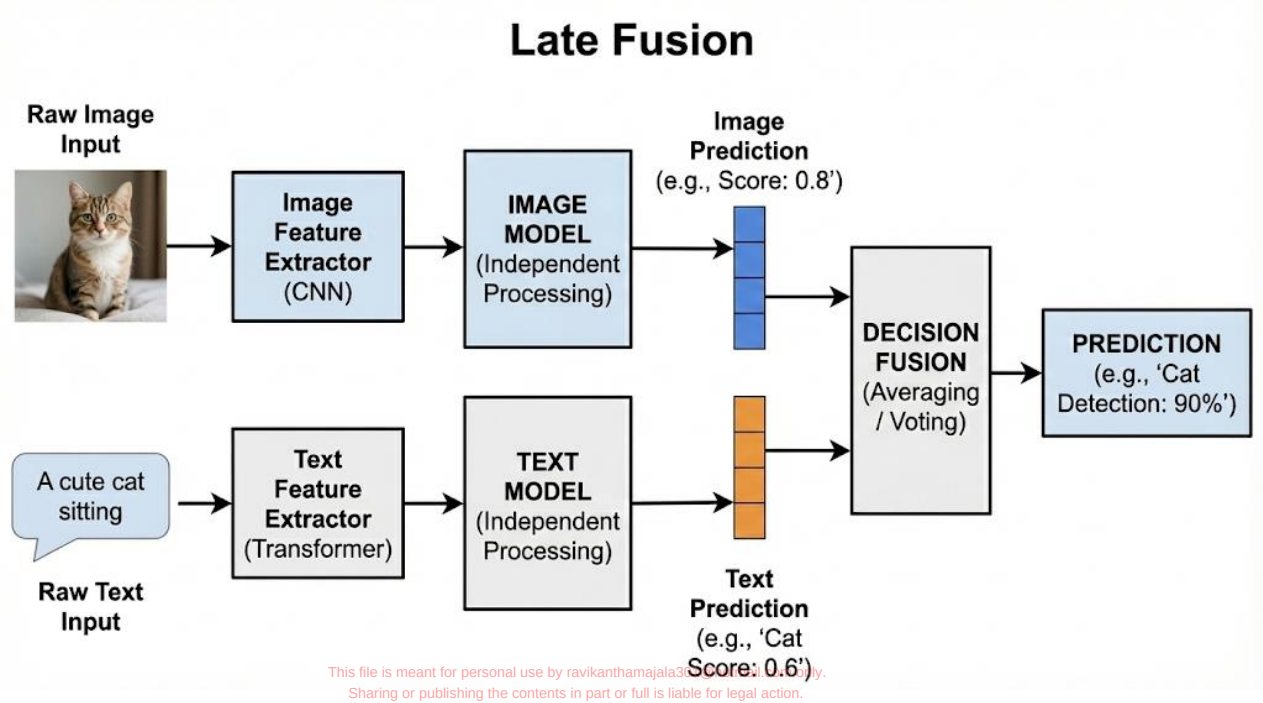
MultiModal Methods

- **Early Fusion:** Combining raw data (pixels + text tokens) at the start.

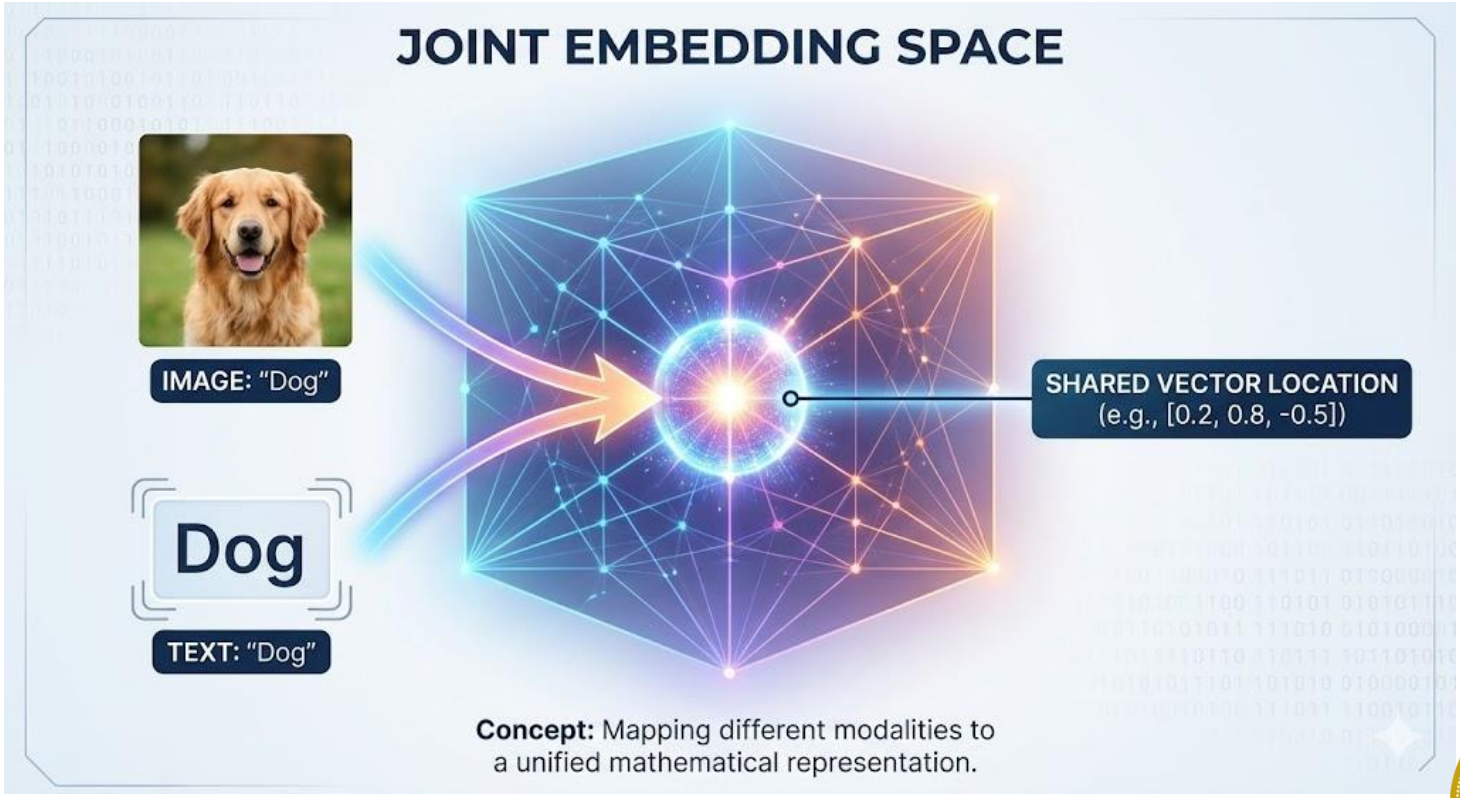


Multimodal Methods

- **Late Fusion:** Processing separately and combining decisions at the end.



Joint embedding Space



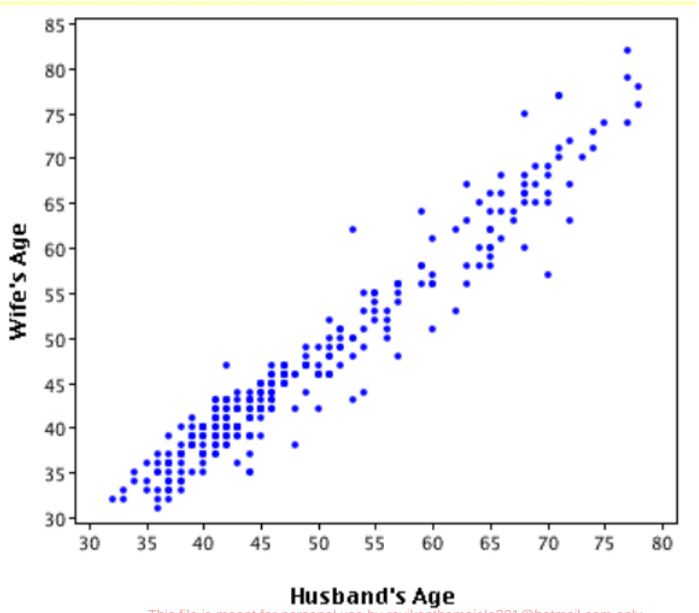
Linear Model

- **What is linear?**
  - It means that you're stating linear relationships between variables, such as
  - ***y = mx + b***

- **What is a linear relationship?**
  - A linear relationship is a statistical term used to describe a straight-line relationship between two variables

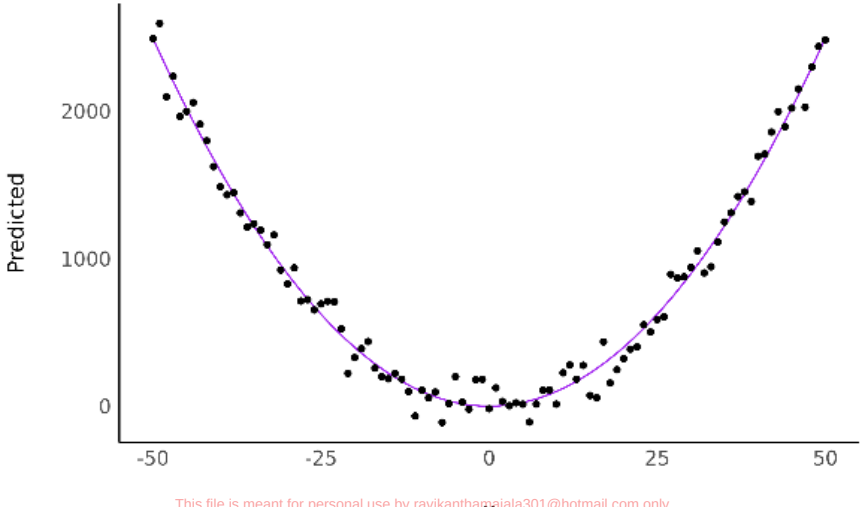
Linear Model

- **What is a linear model?**
  - A linear model, also known as a linear regression model, is a statistical approach used to describe the relationship between a dependent variable and one or more independent variables.



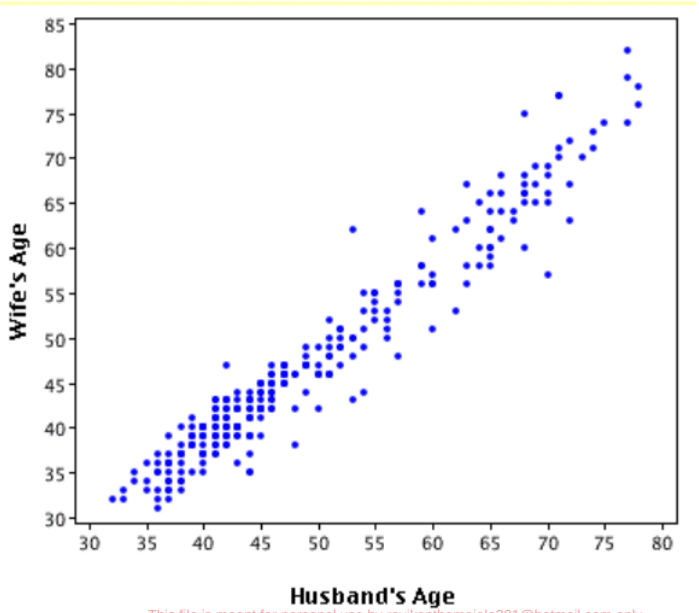
Non-Linear Model?

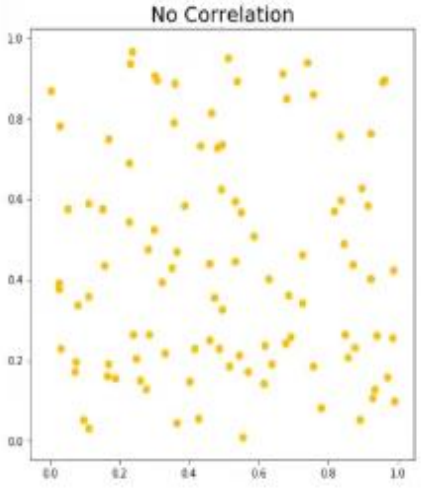
- A non-linear model is also a statistical model that does NOT assume a linear relationship between the dependent variable and the independent variables, meaning that analyzing more complex relationships from more complex and relatively large datasets is available in non-linear models with making curves, exponentials, logarithms, or interactions

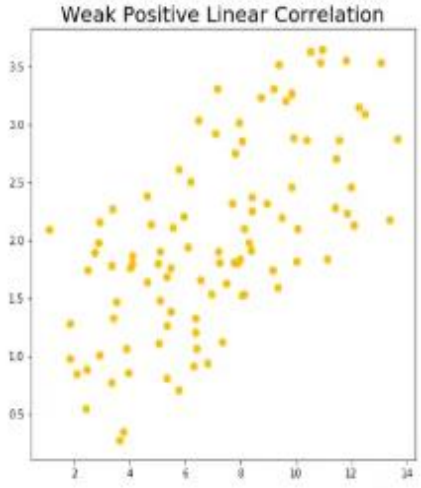


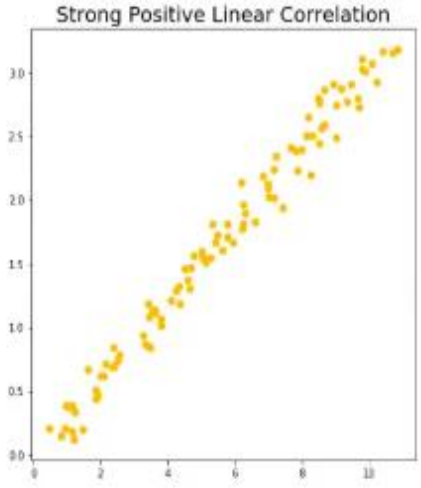
Linear Regression

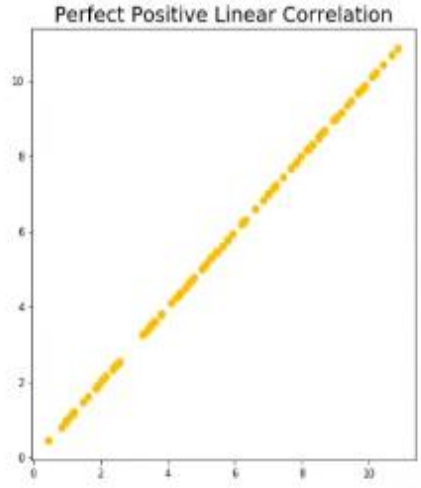
Linear regression analysis is **used to predict the value of a variable based on the value of another variable**

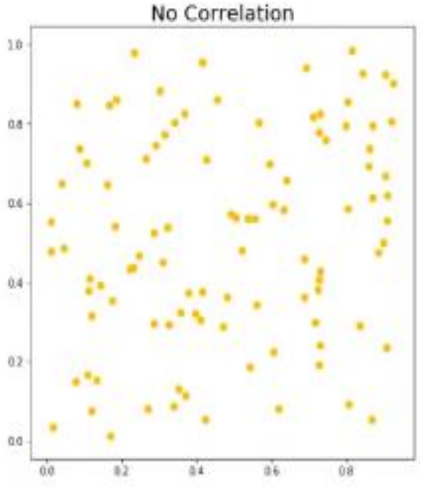


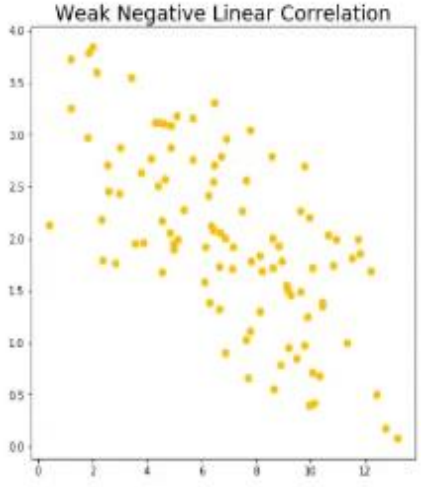


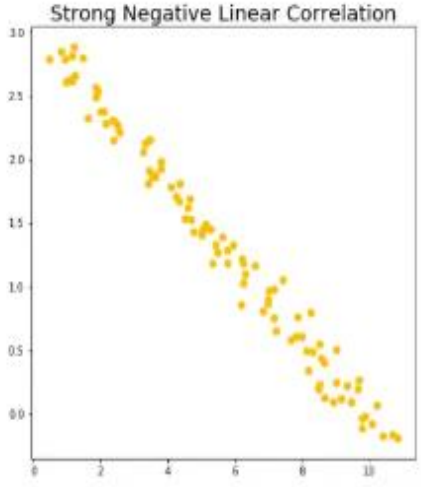


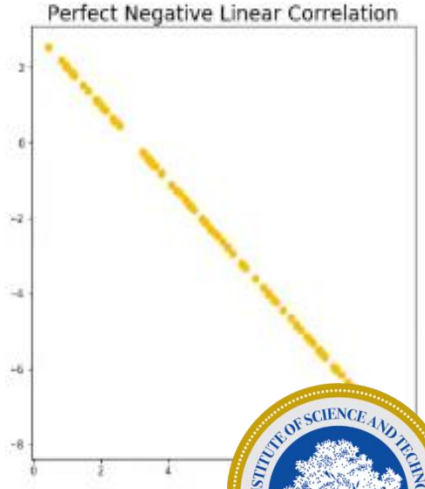












Logistic Regression

- Logistic regression machine learning is a statistical method that is used for building machine learning models where the dependent variable is binary

- Logistic regression is used to describe data and the relationship between one dependent variable and one or more independent variables. The independent variables can be nominal, ordinal, or of interval type.

Logistic Function

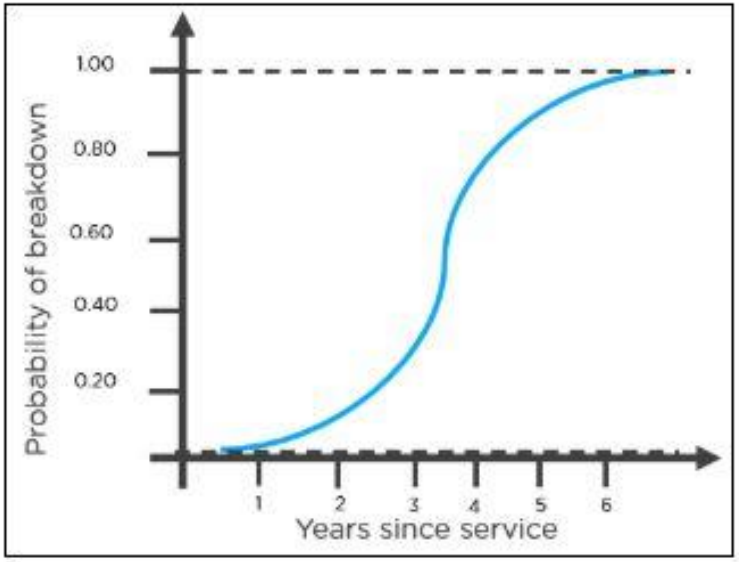
- Logistic regression is named for the function used at the core of the method, the logistic function.

- it’s an S-shaped curve that can take any real-valued number and map it into a value between 0 and 1, but never exactly at those limits.

$$1 / (1 + e^{-value})$$

Logistic Function

- The following is an example of a logistic function we can use to find the probability of a vehicle breaking down, depending on how many years it has been since it was serviced last.

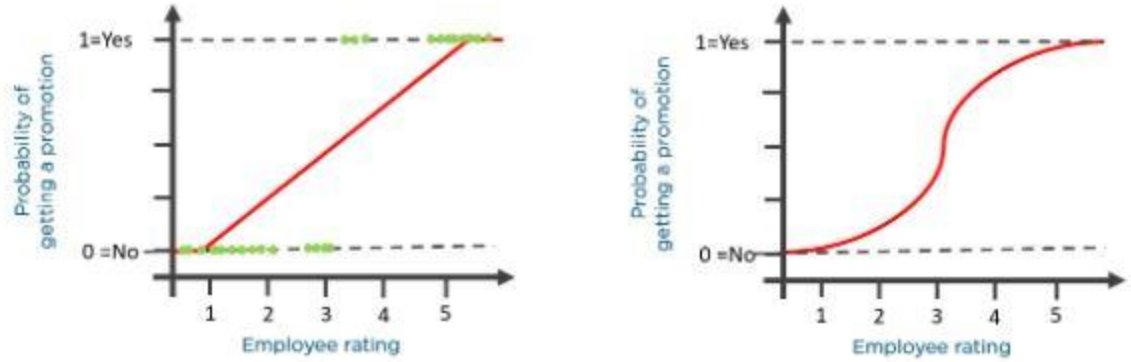


Example of LR

- An organization wants to determine an employee's salary increase based on their performance.
- In this case a linear regression algorithm will help them decide.

Example of LR

- Now, what if the organization wants to know whether an employee would get a promotion or not based on their performance? The above linear graph won't be suitable in this case. As such, we clip the line at zero and one, and convert it into a sigmoid curve (S curve).



Support Vector Machine(SVM) Algorithm

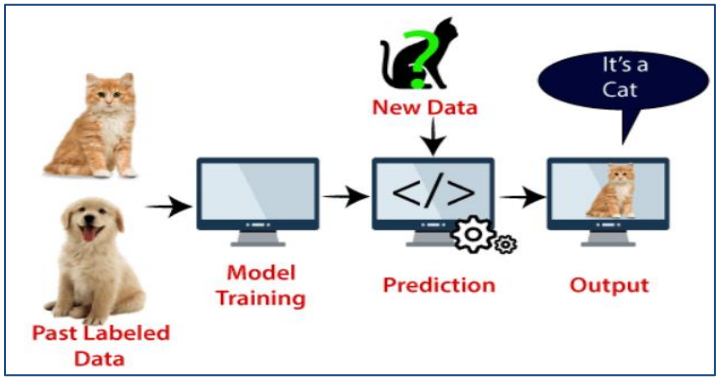
- Used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

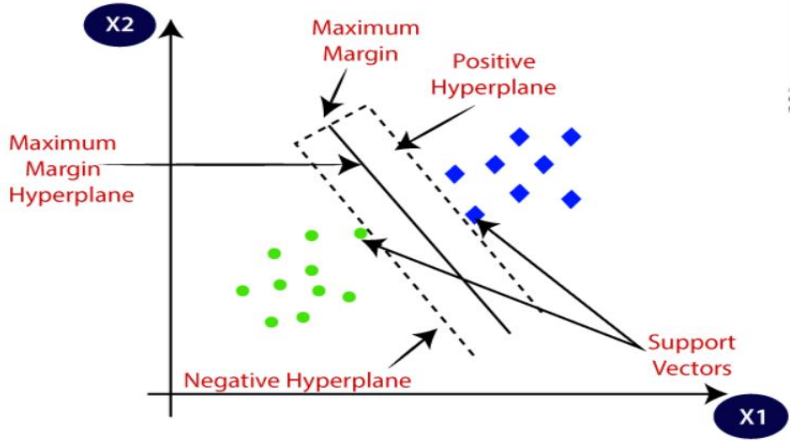
- Goal is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

- SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine.

Support Vector Machine Algorithm

Two different categories that are classified using a decision boundary or hyperplane:





Eg: **SVM algorithm can be used for Face detection, image classification, text categorization, etc.**

Support Vector Machine Algorithm

Types of SVM:

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

Support Vector Machine Algorithm

Hyperplane:

- There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.
- The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.
- We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.

Support Vectors:

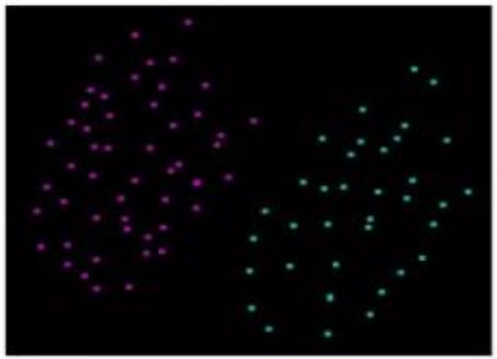
- The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.

Support Vector Machine Algorithm

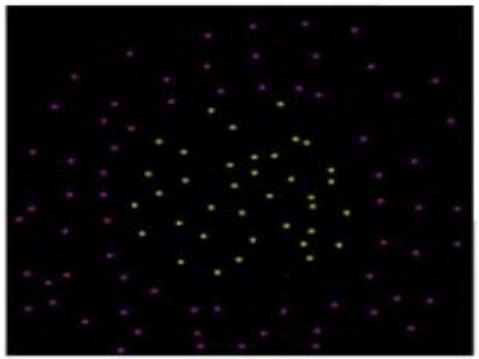
Linearly separable data

A line drawn between these data points classify the black dots and blue squares.

Linear vs Nonlinear separable data



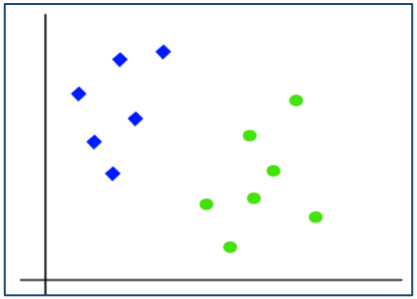
Linearly separable data

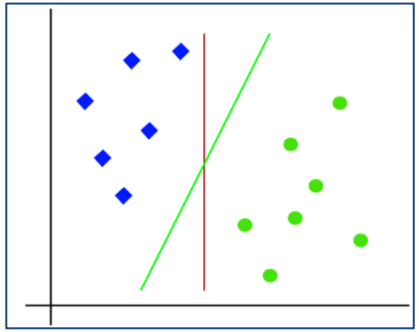


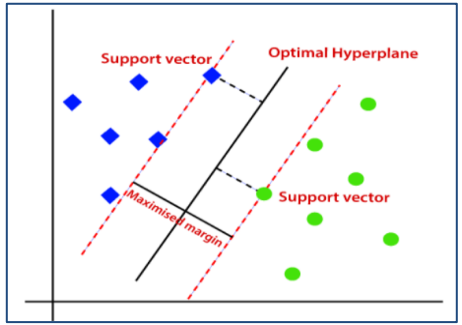
Non linearly separable data

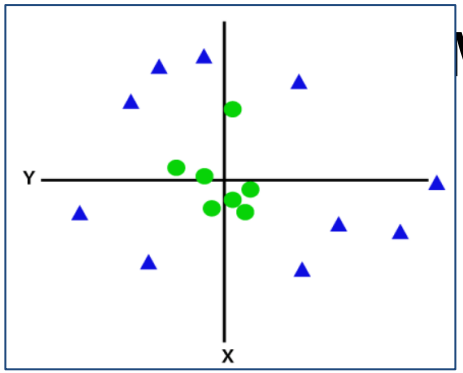
Support Vector Machine Algorithm

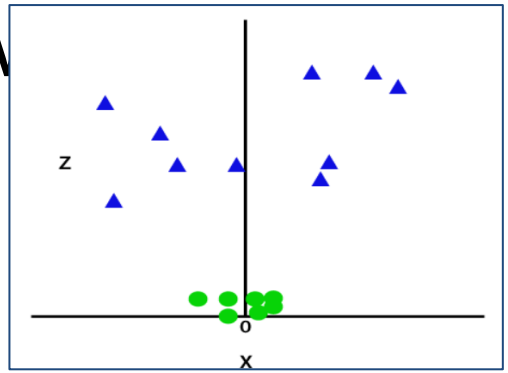
Linear SVM:

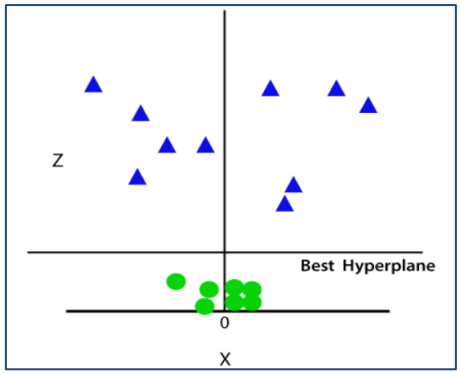


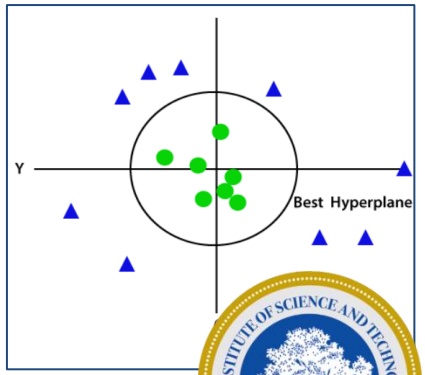












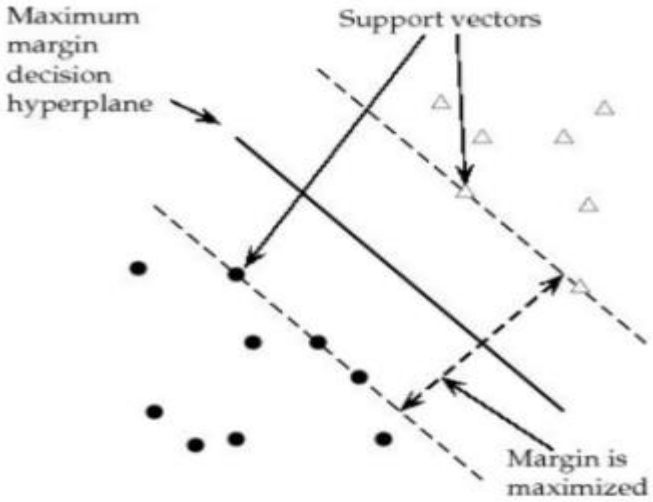
Support Vector Machine Algorithm

Tuning Parameters

Kernel , Regularization, Gamma, Margin

Margin

Margin is the perpendicular distance between the closest data points and the Hyperplane (on both sides) The best optimized line (hyperplane) with maximum margin is termed as Margin Maximal Hyperplane. The closest points where the margin distance is calculated are considered as the support vectors.



Support Vector Machine Algorithm

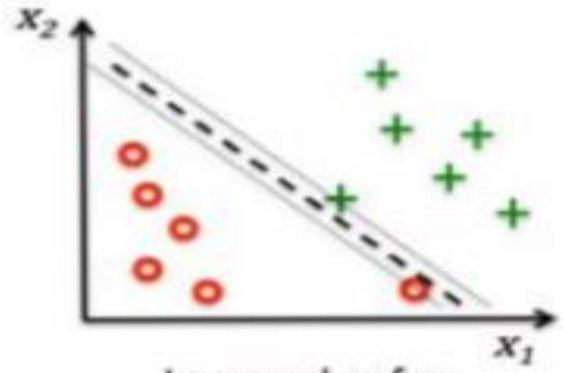
Regularization

- Also the 'C' parameter in Python's SkLearn Library
- Optimises SVM classifier to avoid misclassifying the data.
- C→ large Margin of hyperplane → small
- C→ small Margin of hyperplane → large
- misclassification(possible)

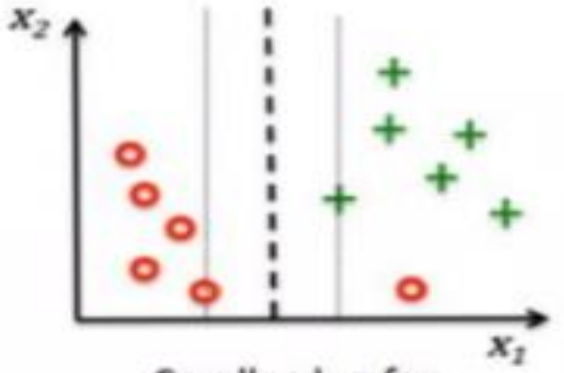
    1. C--> large, chance of overfit
    2. C--> small, chance of underfitting

Support Vector Machine Algorithm

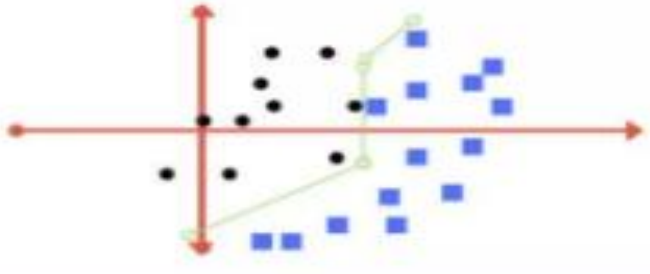
Regularization

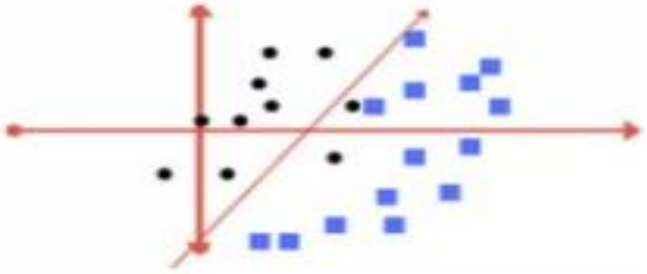


Large value for parameter C



Small value for parameter C





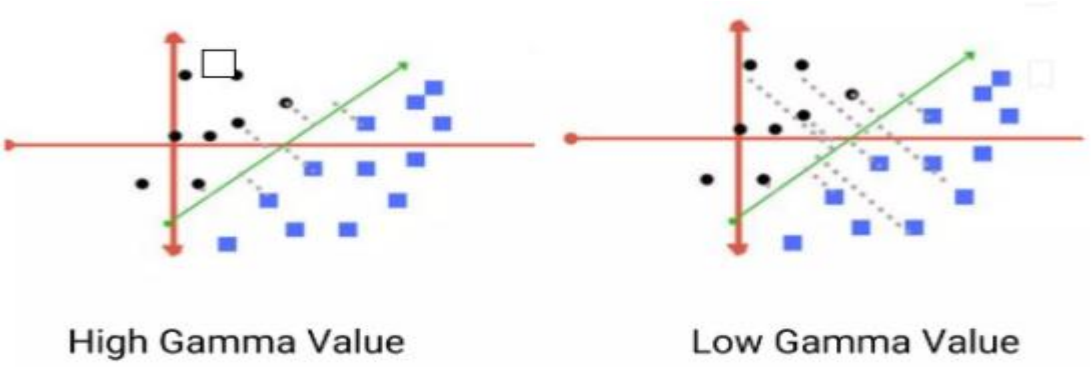
Support Vector Machine Algorithm

Gamma

Defines how far influences the calculation of of plausible line of separation.

• Low gamma ---> points far from plausible line are considered for calculation

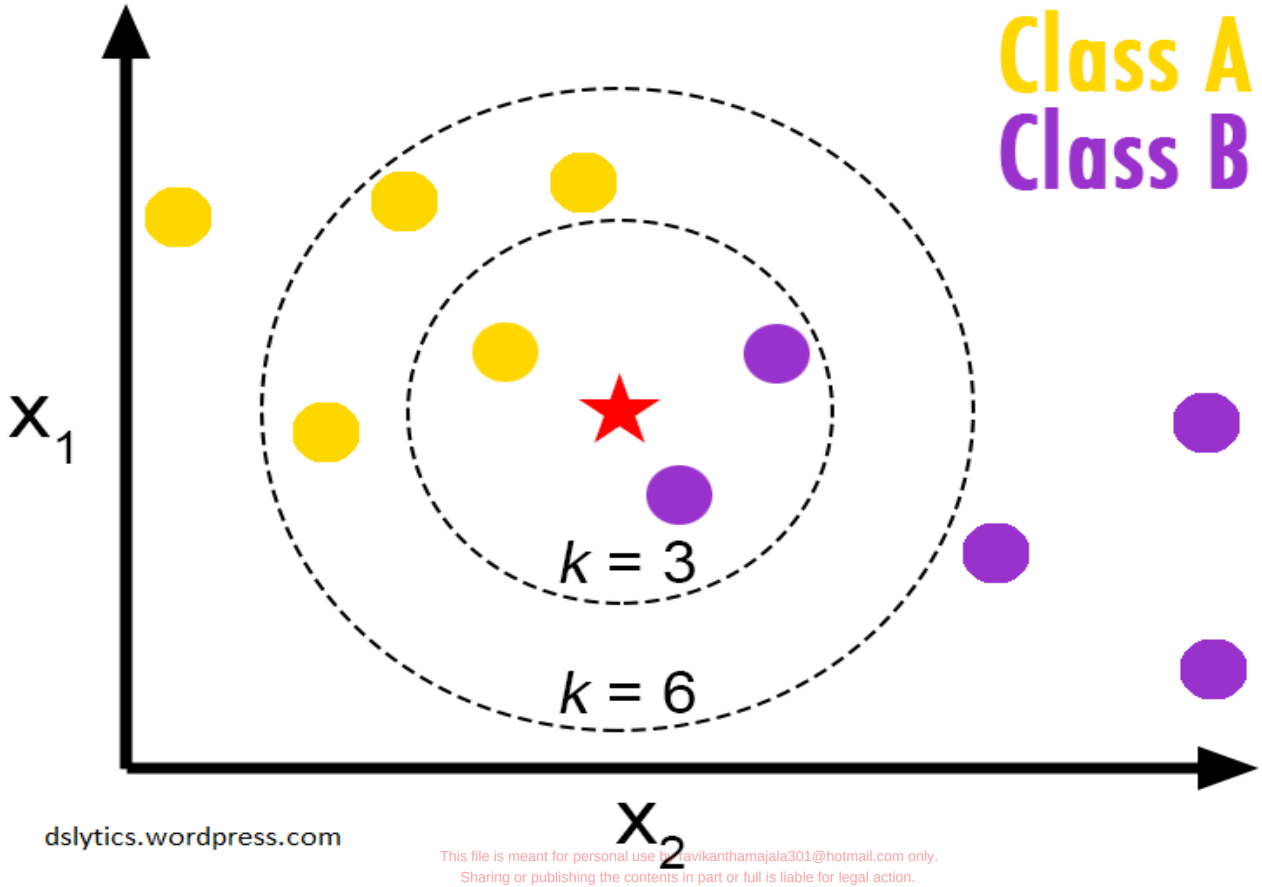
• High gamma ---> points close to plausible line are considered for calculation



Non linear model

- Knn
- Decision tree
- Random Forest
- Naïve Bayes
- Etc

K Nearest Neighbor

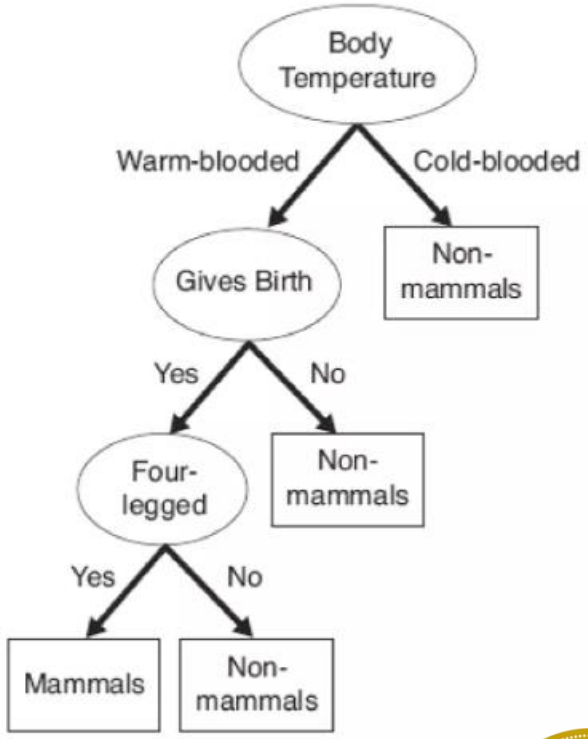


Decision Tree

- A Decision tree is a tree in which each branch node represents a choice between a number of alternatives, and each leaf node represents a decisions

- Root and internal nodes test rules. Leaf nodes make predictions

- Decision Tree (DT) learning is about learning such a tree from labeled data

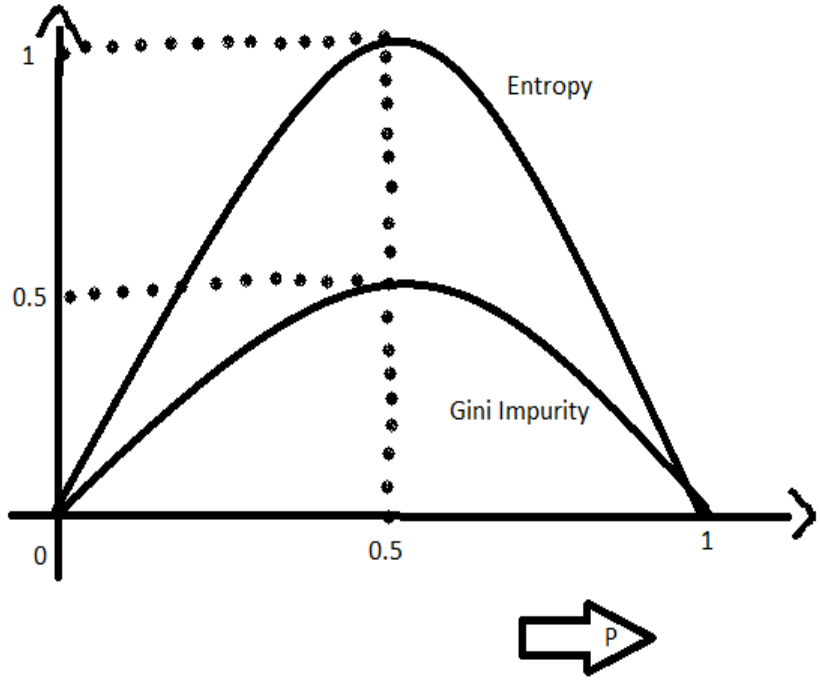


Metrics to Impurity

- GINI
- Entropy
- Variance

Entropy vs Gini

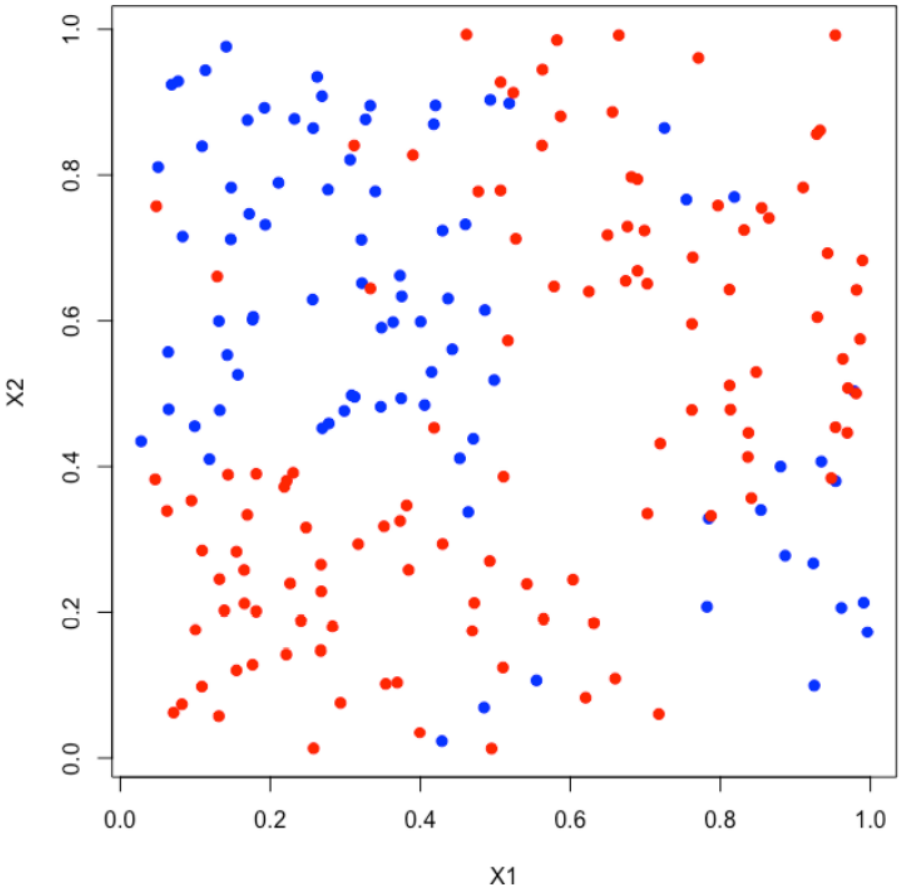
Can be computed by summing the probability of an item with label i being chosen (Pi), times the probability of a mistake (1—Pi) in categorizing that item

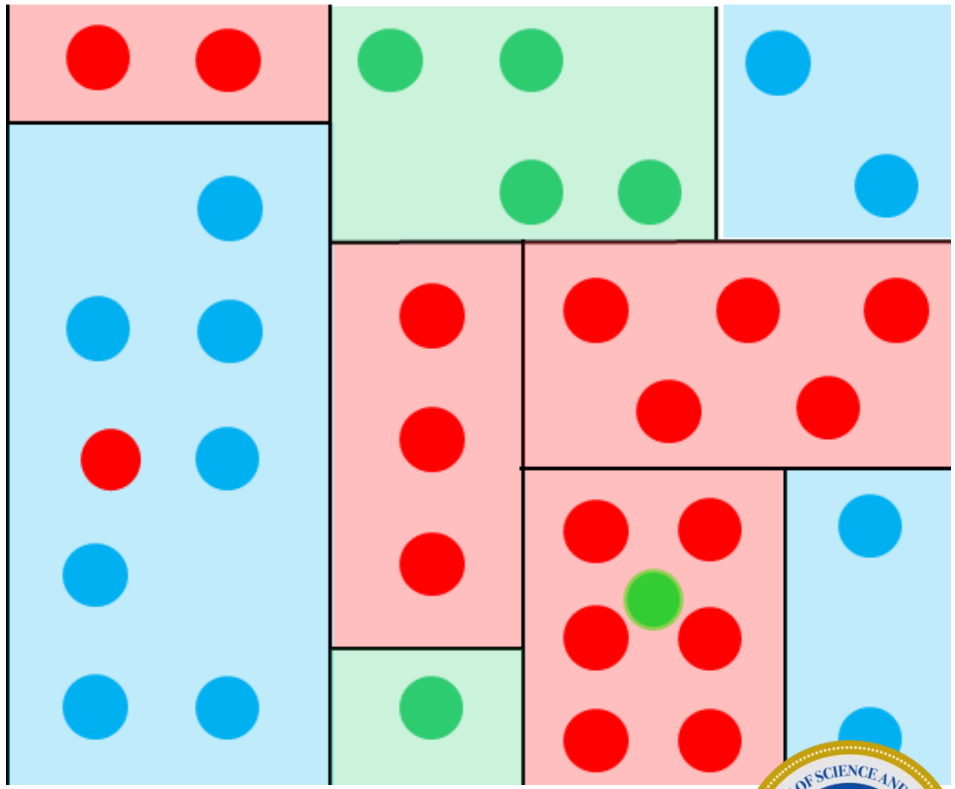


$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

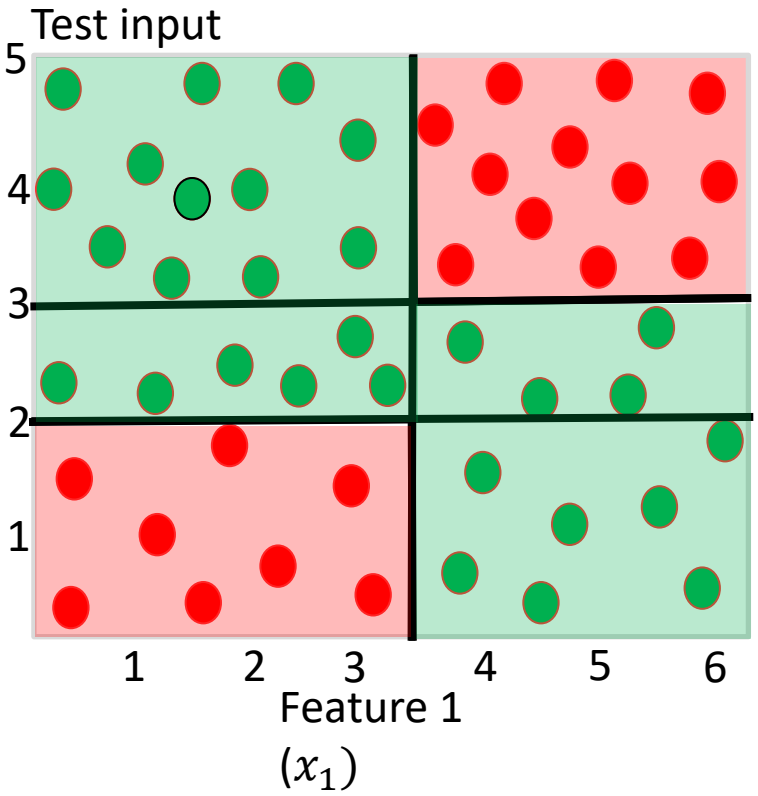
$$Gini(E) = 1 - \sum_{j=1}^{c} p_j^2$$

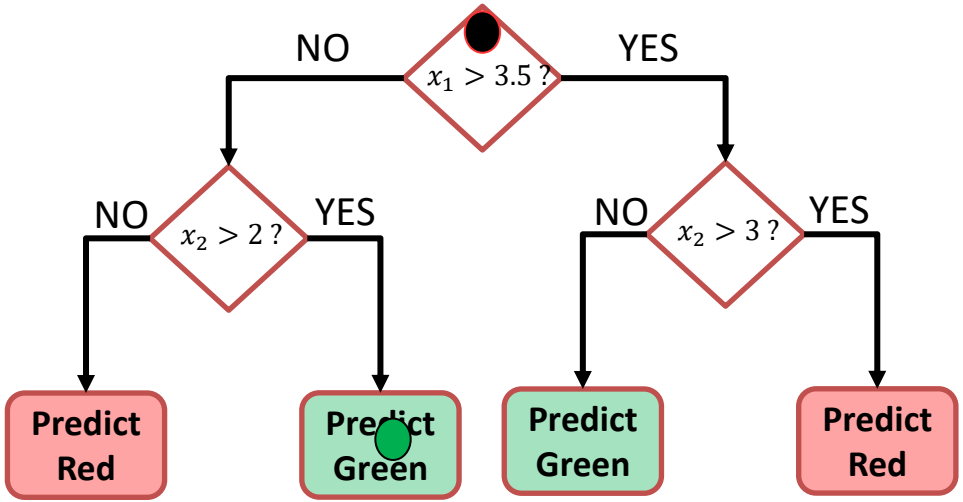
Decision Trees





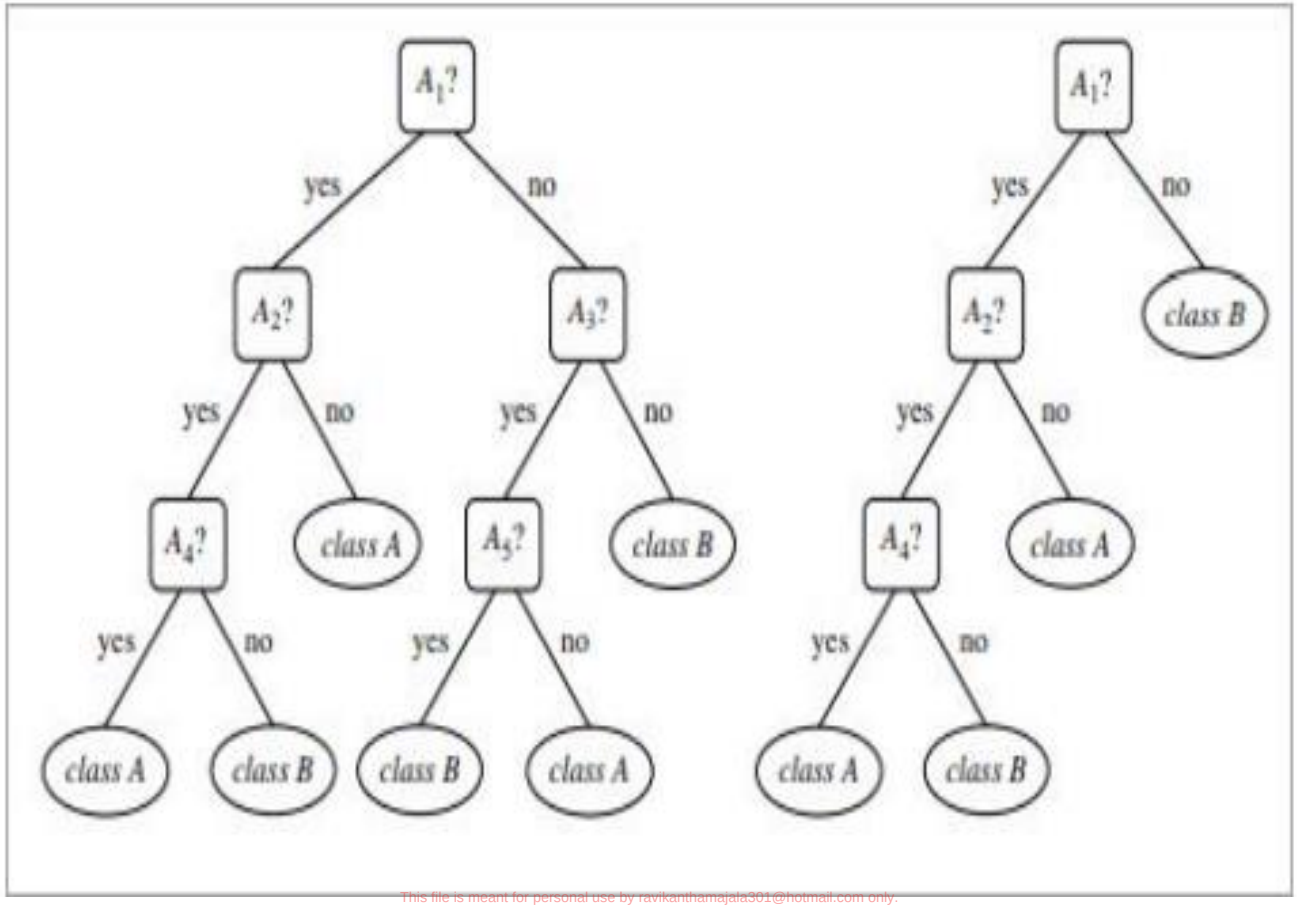
Decision Trees for Classification





Root node contains all training inputs Each leaf node receives a subset of train

Pruning



Pruning

Pre-Pruning (Early Stopping): Stops the tree from growing deeper by setting limits during training, such as:

- Maximum depth of the tree.
- Minimum samples required to split a node.
- Maximum number of features to consider for a split.

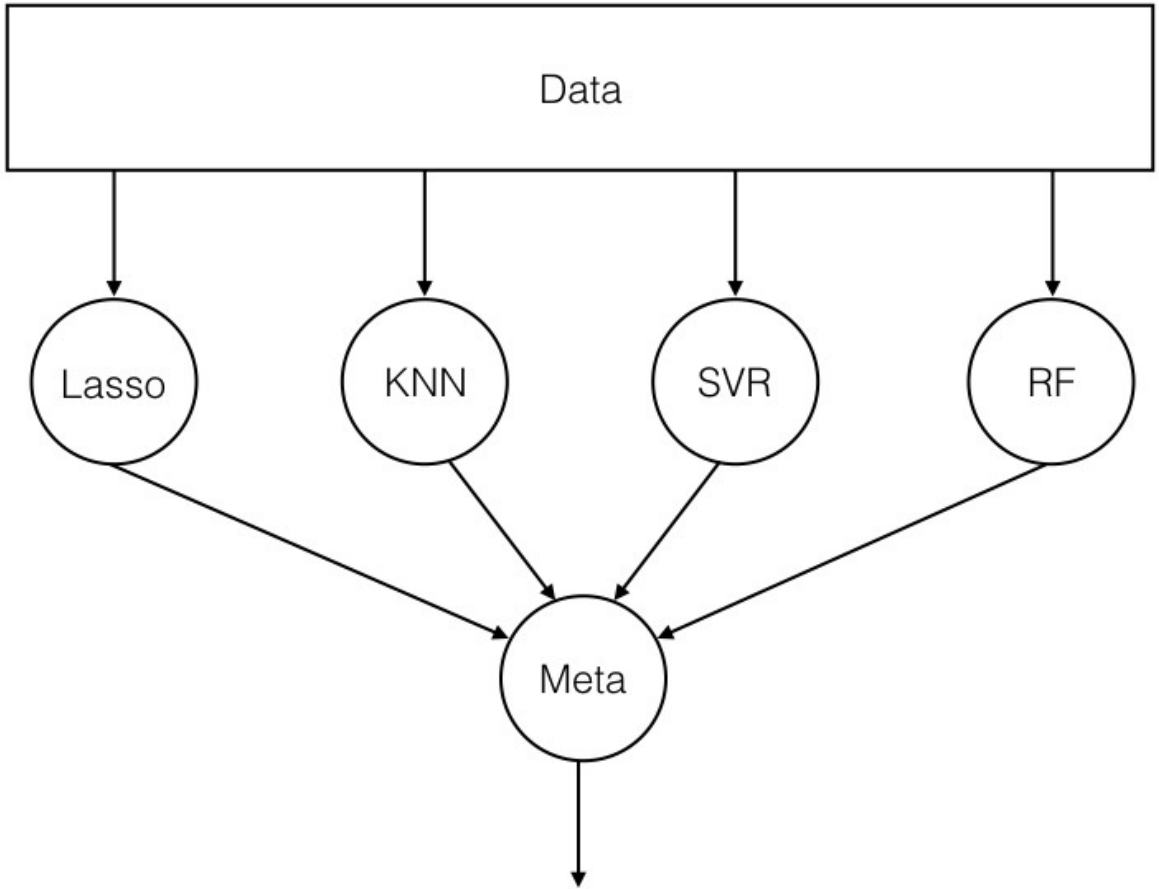
Ensemble Learning

- An ensemble is a composite model, combines a series of low performing classifiers with the aim of creating an improved classifier.
- Here, individual classifier vote and final prediction label returned that performs majority voting.
- Ensembles offer more accuracy than individual or base classifier.
- Ensemble methods can parallelize by allocating each base learner to different-different machines.
- Finally, you can say Ensemble learning methods are meta-algorithms that combine several machine learning methods into a single predictive model to increase performance.
- Ensemble methods can decrease variance using bagging approach, bias using a boosting approach, or improve predictions using stacking approach.

Ensembling

- An ensemble is the art of combining a diverse set of learners (individual models) together to improvise on the stability and predictive power of the model.
- In our example, the way we combine all the predictions collectively will be termed as Ensemble learning.
- Moreover, Ensemble-based models can be incorporated in both of the two scenarios, i.e., when data is of large volume and when data is too little.

Basic Ensemble Structure



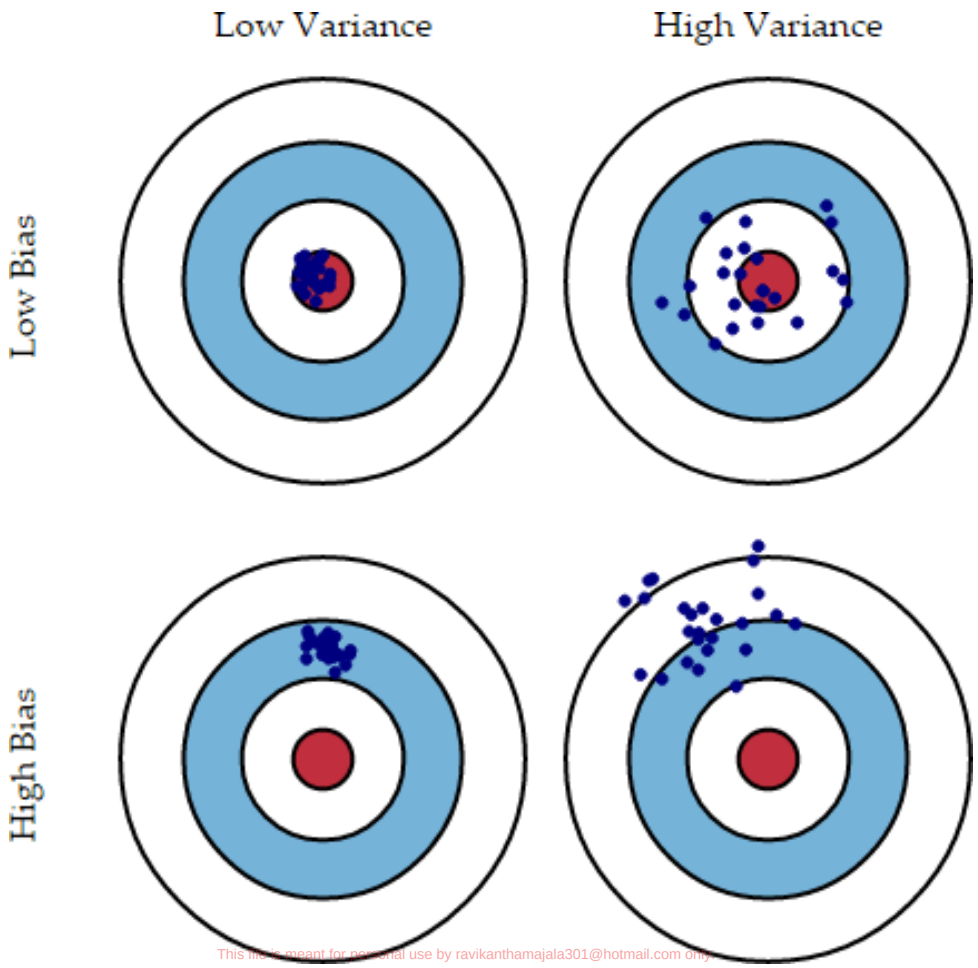
Why ensembles ?

- There are two main reasons to use an ensemble over a single model, and they are related; they are:
- Performance: An ensemble can make better predictions and achieve better performance than any single contributing model.
- Robustness: An ensemble reduces the spread or dispersion of the predictions and model performance.

Model Error

- Bias error is useful to quantify how much on an average are the predicted values different from the actual value. A high bias error means we have an under-performing model which keeps on missing essential trends.
- Variance on the other side quantifies how are the prediction made on the same observation different from each other. A high variance model will over-fit on your training population and perform poorly on any observation beyond training.

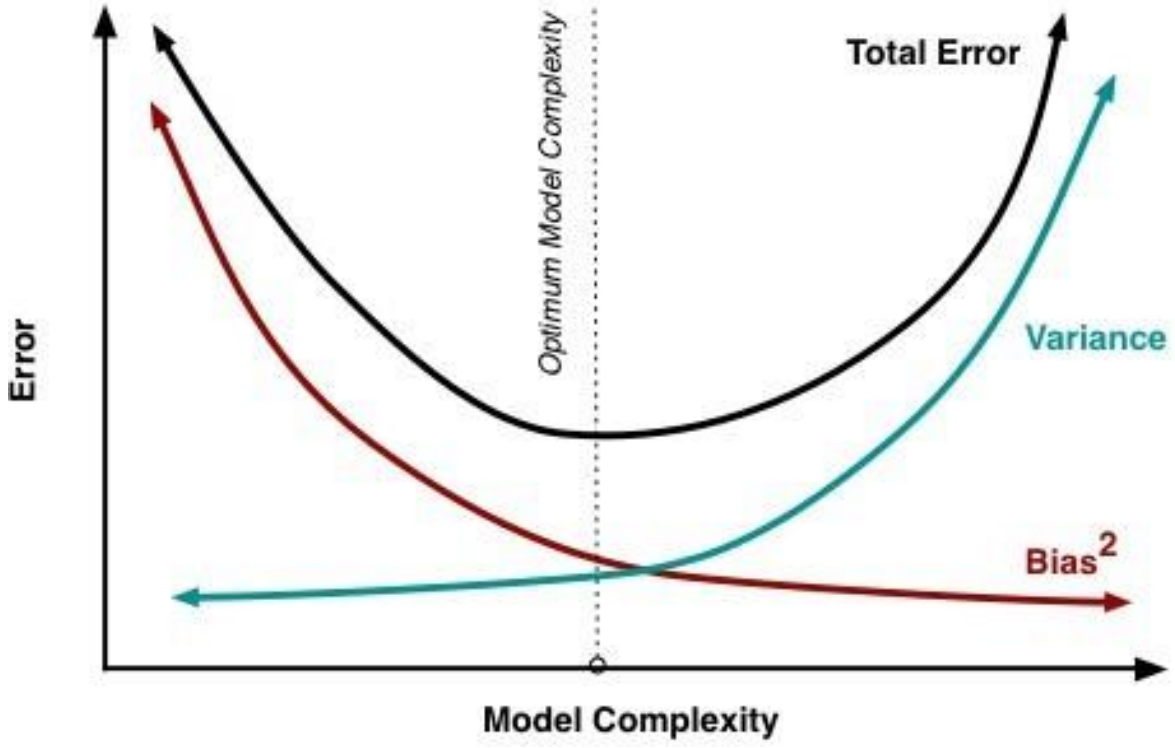
Bias and Variance



Bias – Variance Tradeoff

- Normally, as you increase the complexity of your model, you will see a reduction in error due to lower bias in the model. However, this only happens till a particular point.
- As you continue to make your model more complex, you end up over-fitting your model and hence your model will start suffering from high variance.
- A champion model should maintain a balance between these two types of errors. This is known as the trade-off management of bias-variance errors. Ensemble learning is one way to execute this trade off analysis.

Bias – Variance Tradeoff



Bias – Variance Tradeoff

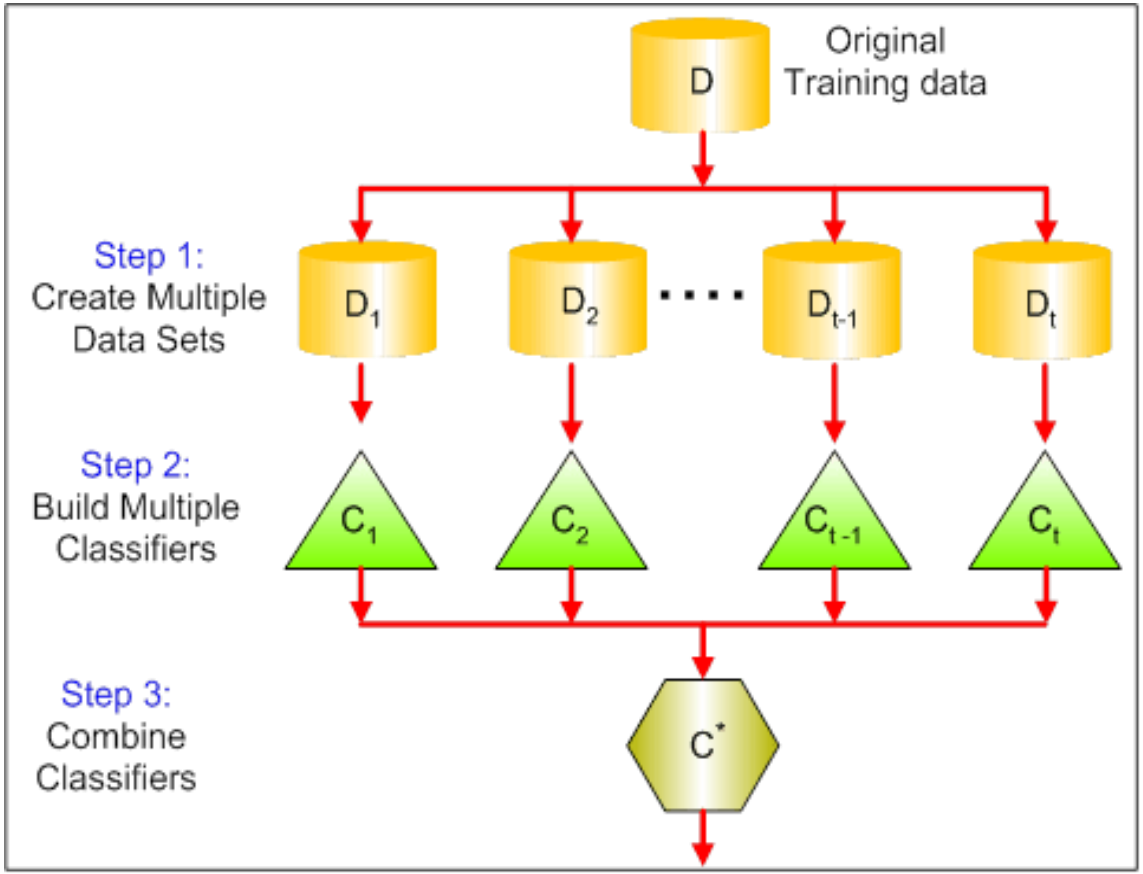
Ensemble Creation Approaches

- Unweighted Voting (e.g. Bagging)
- Weighted voting – based on accuracy (e.g. Boosting), Expertise, etc.
- Stacking - Learn the combination function

Bagging

- Bagging stands for bootstrap aggregation.
- It combines multiple learners in a way to reduce the variance of estimates.
- For example, random forest trains M Decision Tree, you can train M different trees on different random subsets of the data and perform voting for final prediction.
- Example:
- Random Forest
- Extra Trees.

Bagging



Boosting

- Boosting algorithms are a set of the low accurate classifier to create a highly accurate classifier.
- Low accuracy classifier (or weak classifier) offers the accuracy better than the flipping of a coin.
- This is done by building a model from the training data, then creating a second model that attempts to correct the errors from the first model. Models are added until the training set is predicted perfectly or a maximum number of models are added.
- Highly accurate classifier(or strong classifier) offer error rate close to 0. Boosting algorithm can track the model who failed the accurate prediction.
- Boosting algorithms are less affected by the overfitting problem.

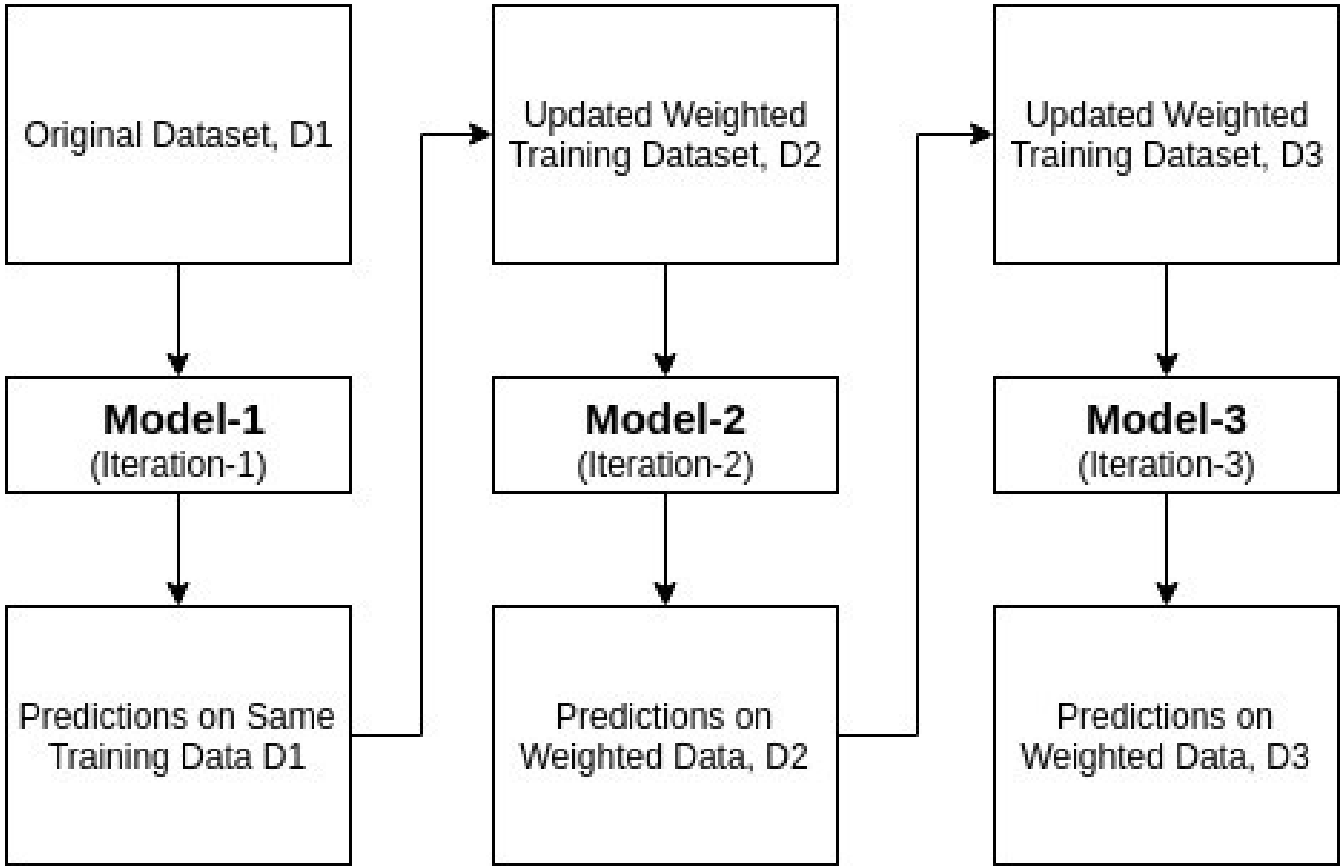
Boosting Models

- Models that are typically used in Boosting technique are:
- XGBoost (Extreme Gradient Boosting)
- GBM (Gradient Boosting Machine)
- ADABoost (Adaptive Boosting)

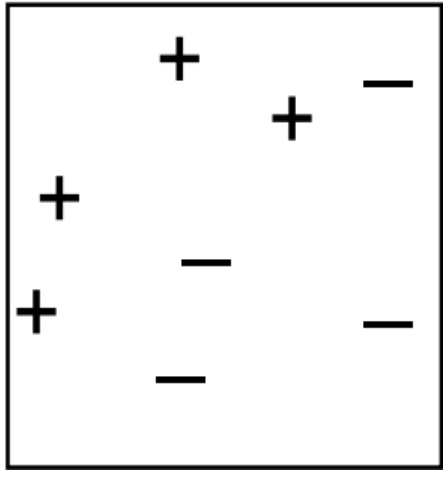
Adaboost Summary

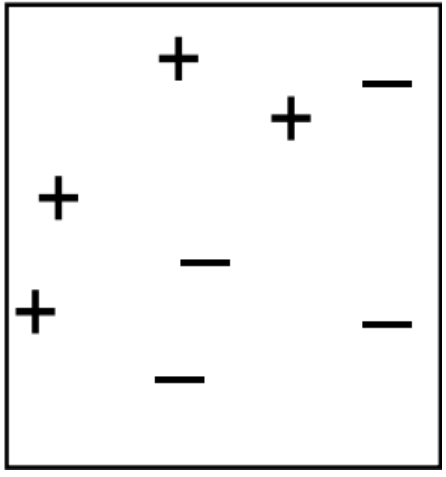
- Initially, Adaboost selects a training subset randomly.
- It iteratively trains the AdaBoost machine learning model by selecting the training set based on the accurate prediction of the last training.
- It assigns the higher weight to wrong classified observations so that in the next iteration these observations will get the high probability for classification.
- Also, It assigns the weight to the trained classifier in each iteration according to the accuracy of the classifier. The more accurate classifier will get high weight.
- This process iterate until the complete training data fits without any error or until reached to the specified maximum number of estimators.
- To classify, perform a "vote" across all of the learning algorithms you built.

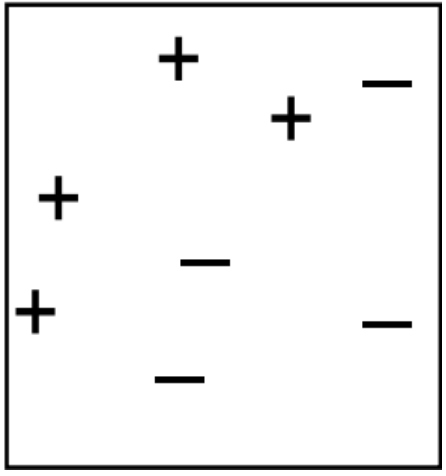
How Adaboost Works?

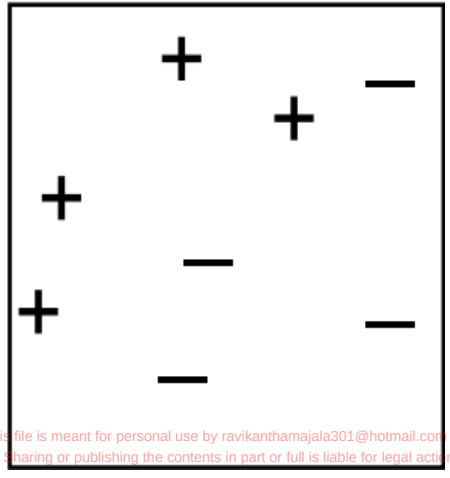


Boosting







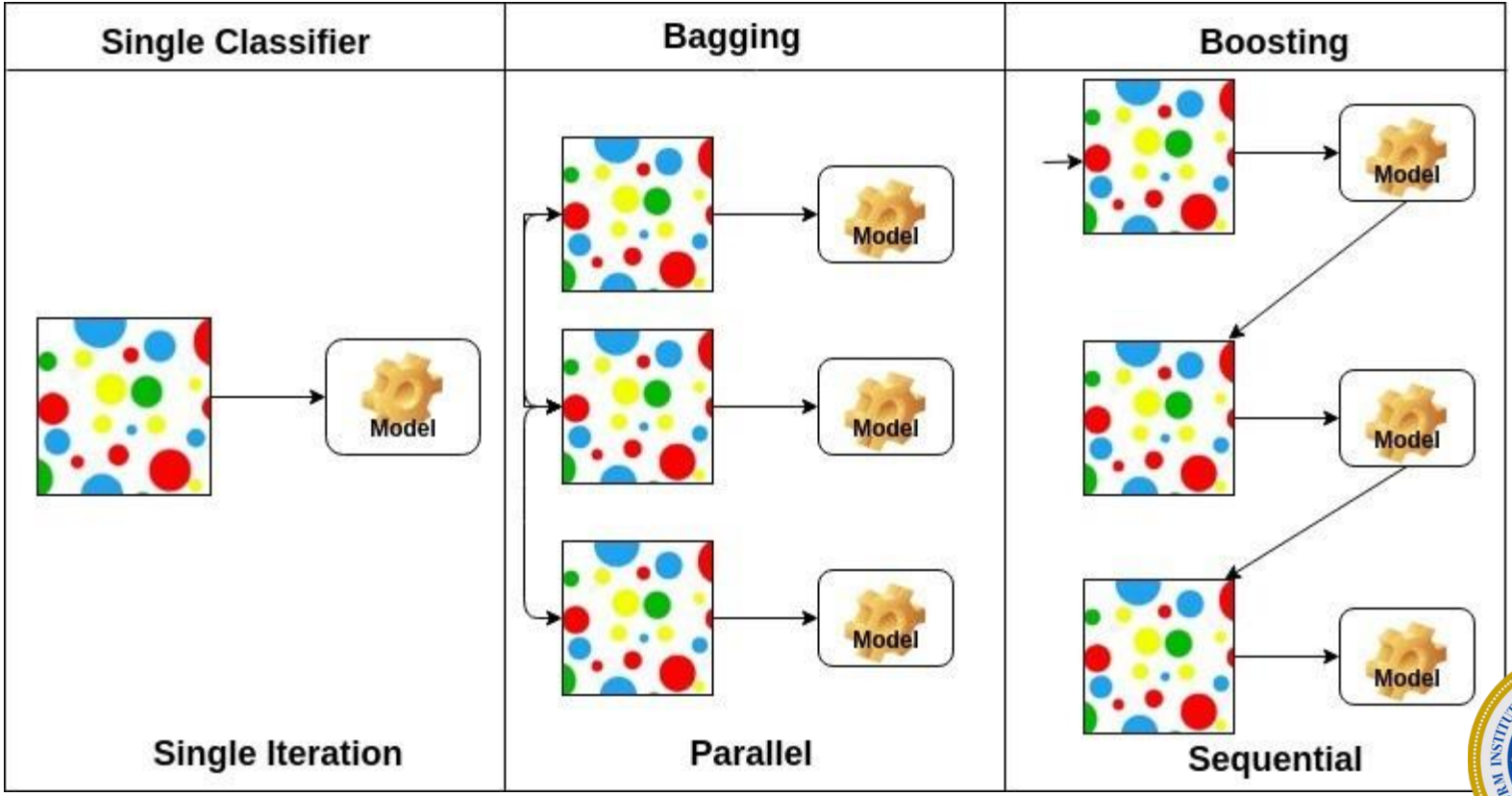


Gradient Boosting

- Gradient Boosting is a popular boosting algorithm. In gradient boosting, each predictor corrects its predecessor's error.

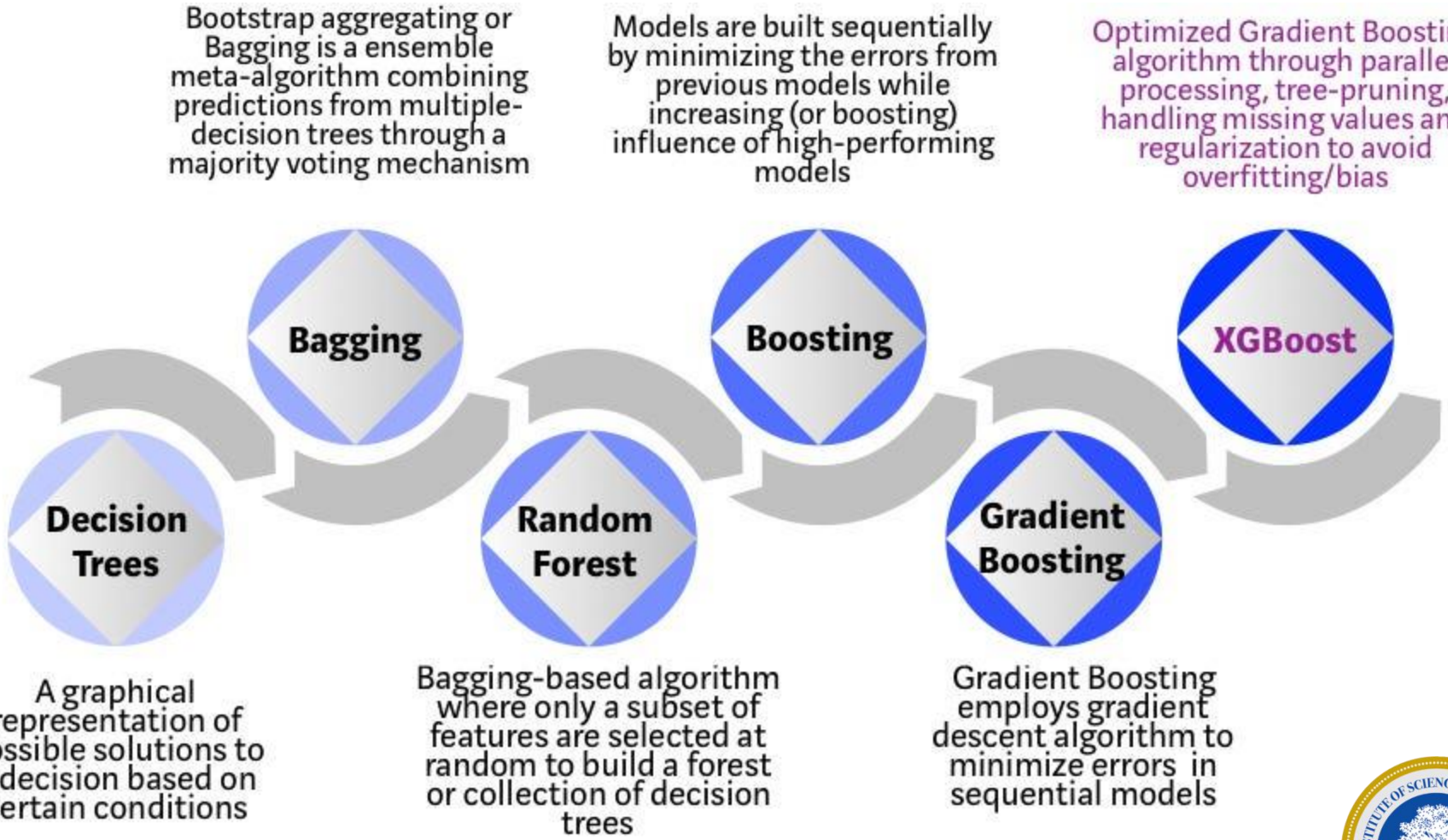
- In contrast to Adaboost, the weights of the training instances are not tweaked, instead, each predictor is trained using the residual errors of predecessor as labels.

Comparison



XG Boost

- Stands for:
- eXtreme Gradient Boosting.
- XGBoost is a powerful iterative learning algorithm based on gradient boosting.
- Robust and highly versatile, with custom objective loss function compatibility.



XG Boost

- Tree-Based Boosting algorithm.
- 4 Critical Parameters for Tuning:
- η: ETA or “Learning Rate”
- max_depth: Controls the “height” of the tree via splits.
- γ: Minimum required loss for the model to justify a split.
- λ: L2 (Ridge) regularization on variable weights.

Advantage

- All of the advantages of gradient boosting, plus more.
- Frequent Kaggle data competition champion.
- Utilizes CPU Parallel Processing by default.
- Two main reasons for use:
- Low Runtime
- High Model Performance

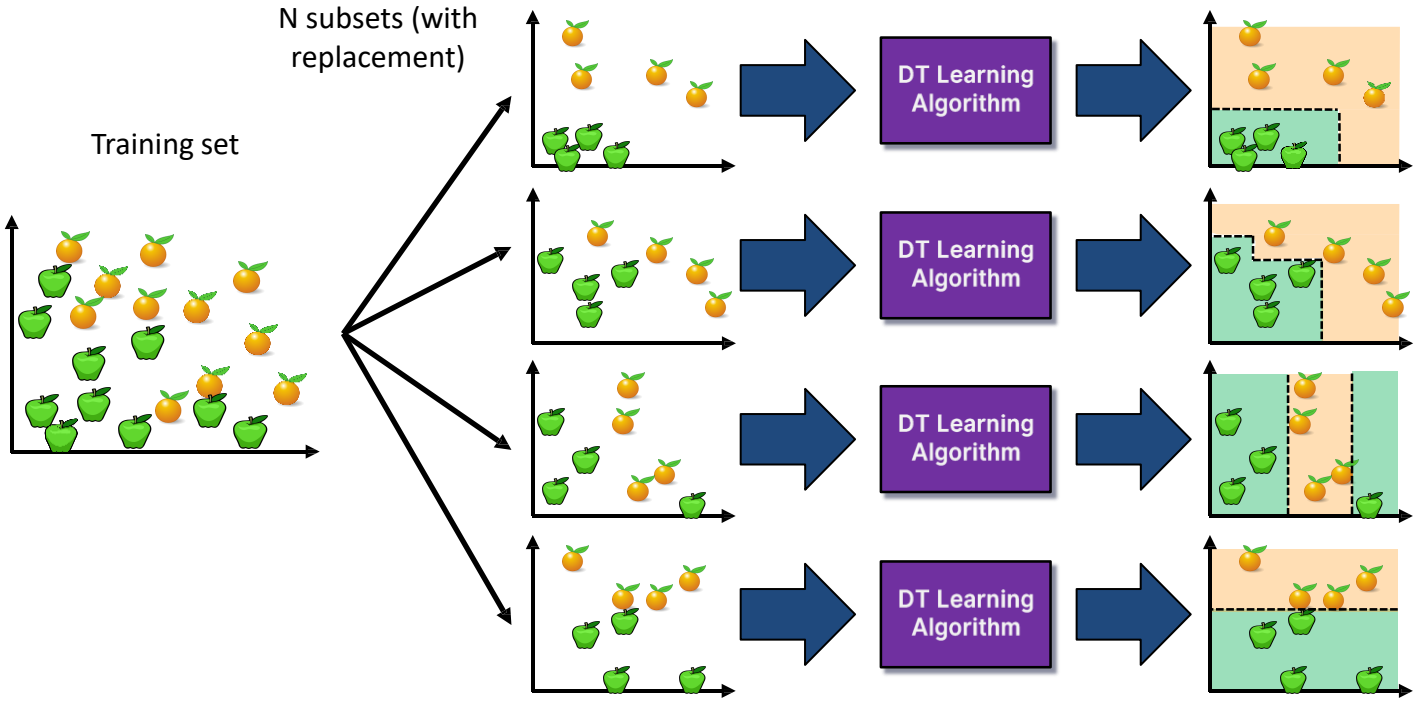
What is a Random Forest?

- We have a single data set, so how do we obtain slightly different trees?

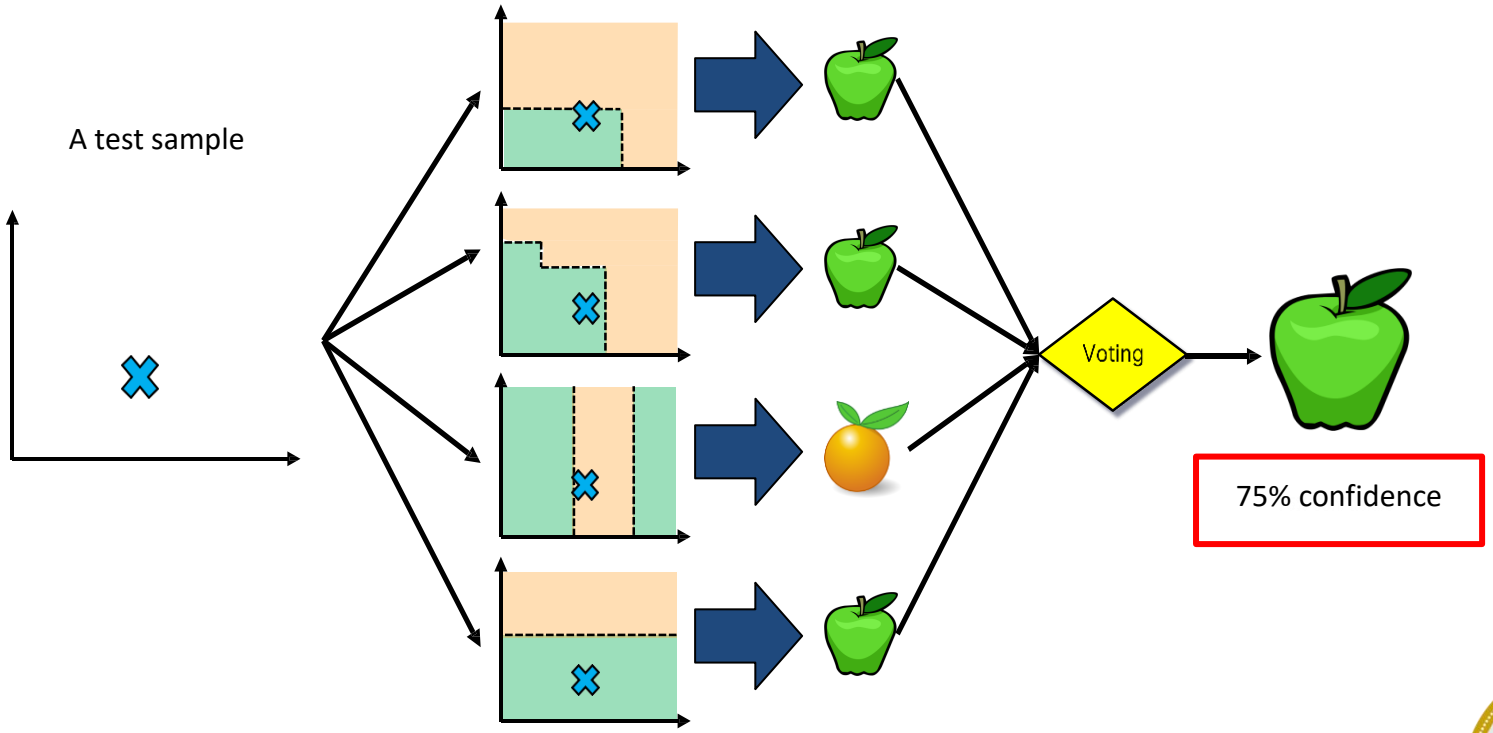
    1. Bagging (Bootstrap Aggregating):
    - Take random subsets of data points from the training set to create N smaller data sets
    - Fit a decision tree on each subset
    - Results in low variance.

    2. Random Subspace Method (also known as Feature Bagging):
    - Fit N different decision trees by constraining each one to operate on a random subset of features

Bagging at training time



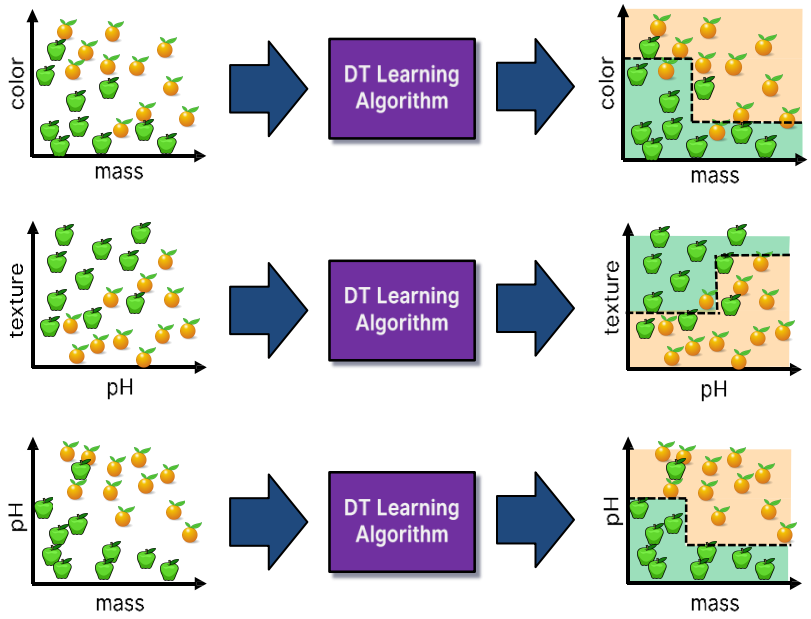
Bagging at inference time



Random Subspace Method at training time

Training data

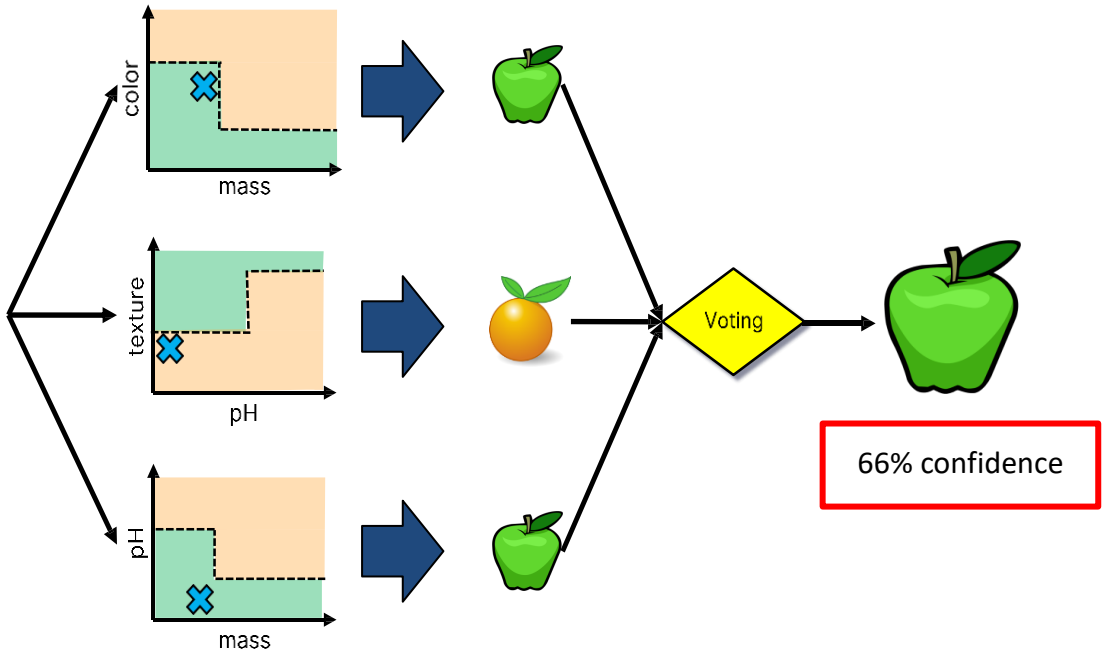
Mass (g)	Color	Texture	pH	Label
84	Green	Smooth	3.5	Apple
121	Orange	Rough	3.9	Orange
85	Red	Smooth	3.3	Apple
101	Orange	Smooth	3.7	Orange
111	Green	Rough	3.5	Apple
...				
117	Red	Rough	3.4	Orange



Random Subspace Method at inference time

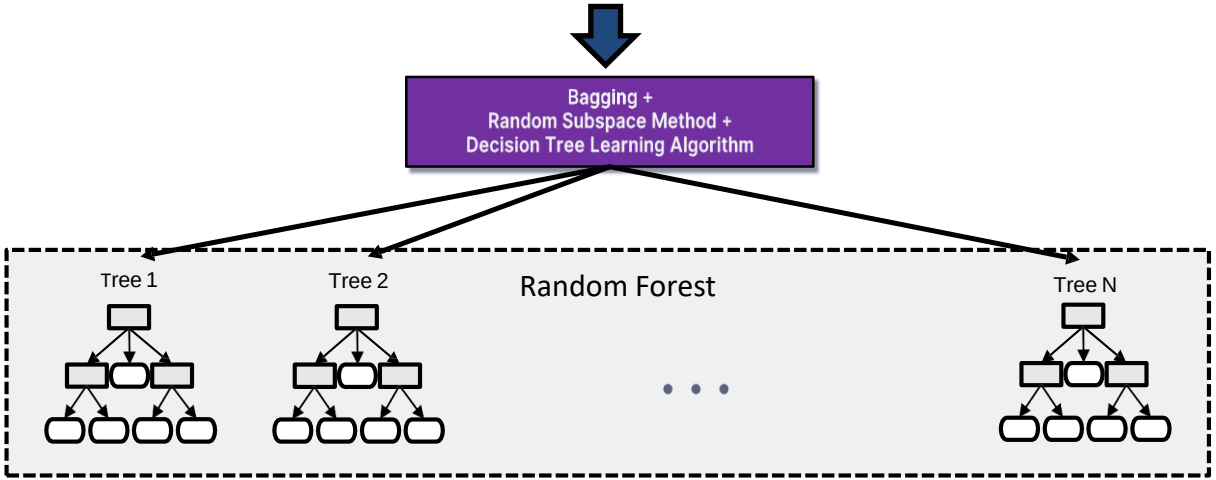
A test sample

87	Red	Smooth	3.1



Random Forests

Mass (g)	Color	Texture	pH	Label
84	Green	Smooth	3.5	Apple
121	Orange	Rough	3.9	Orange
85	Red	Smooth	3.3	Apple
101	Orange	Smooth	3.7	Orange
111	Green	Rough	3.5	Apple
...				
117	Red	Rough	3.4	Orange



Random Forests Algorithm

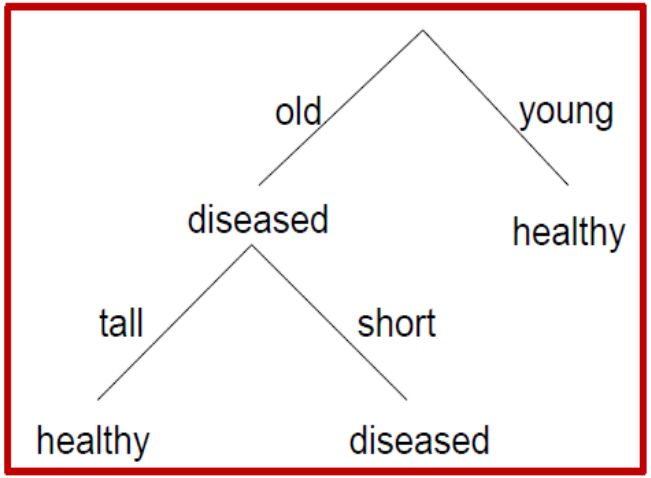
For b = 1 to B:

- (a) Draw a bootstrap sample Z∗ of size *N* from the training data.
- (b) Grow a random-forest tree to the bootstrapped data, by recursively repeating the following steps for each terminal node of the tree, until the minimum node size $\boldsymbol{n_{min}}$ is reached.
  - i. Select ***m*** variables at random from the *p* variables.
  - ii. Pick the best variable/split-point among the *m*.
  - iii. Split the node into two daughter nodes.
- Output the ensemble of trees.

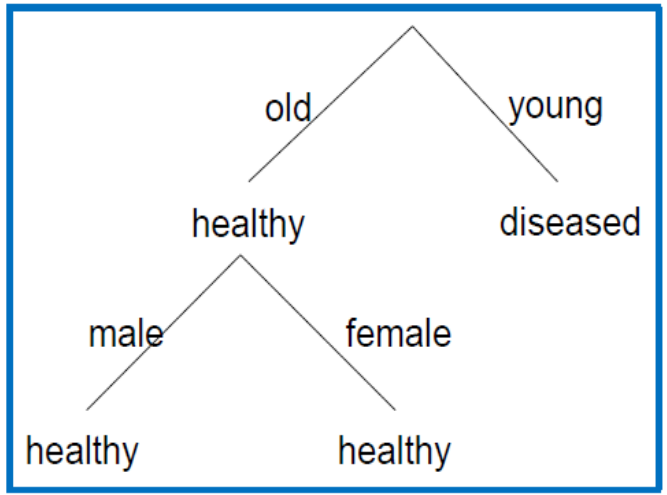
To make a prediction at a new point *x* we do:

- For regression: average the results
- For classification: majority vote

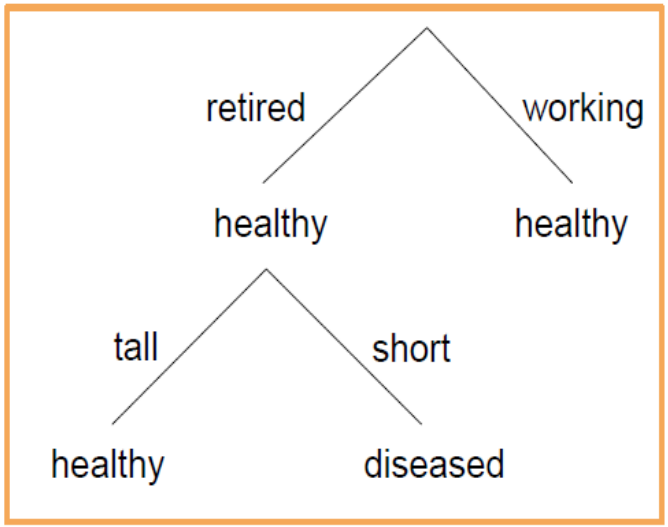
Tree 1



Tree 2



Tree 3



New sample:

old, retired, male, short

Tree predictions:

diseased, healthy, diseased

Majority rule:

diseased

Why Random Forests works:

- Mean squared Error = Variance + Bias
- If trees are sufficiently deep, they have very small bias
- The variance is $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. Hence, if the number of trees increases, the variance decreases

Advantages of Random Forest

- No need for pruning trees
- Accuracy and variable importance generated automatically
- Overfitting is not a problem
- Not very sensitive to outliers in training data
- Easy to set parameters
- Good performance

Regularization

The Problem: Overfitting (The "Memorizer")

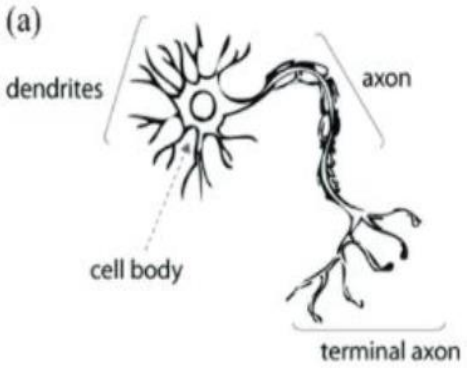
- Imagine a student studying for a math exam.
- **A good student (Generalization):** Learns the formulas and concepts. They can solve *new* problems they haven't seen before.
- **A bad student (Overfitting):** Memorizes the exact answers to the practice questions. If you change one number on the test, they fail.
- In Machine Learning, **Regularization** stops the model from being the "bad student." It forces the model to be simple, preventing it from memorizing noise.

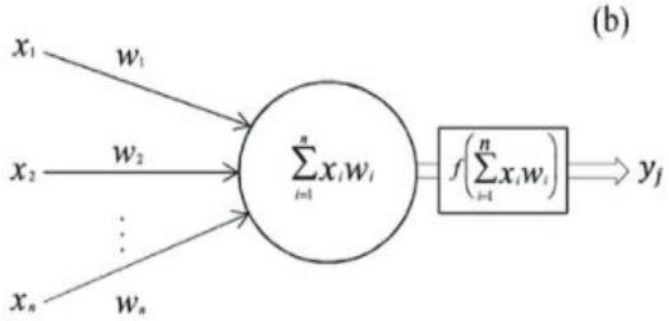
Regularization

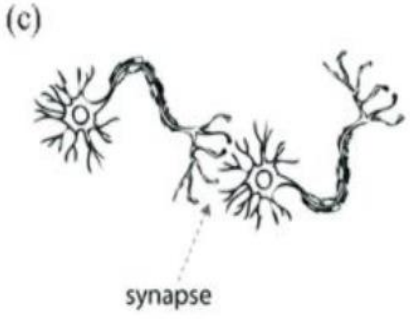
The Solution: The "Penalty"

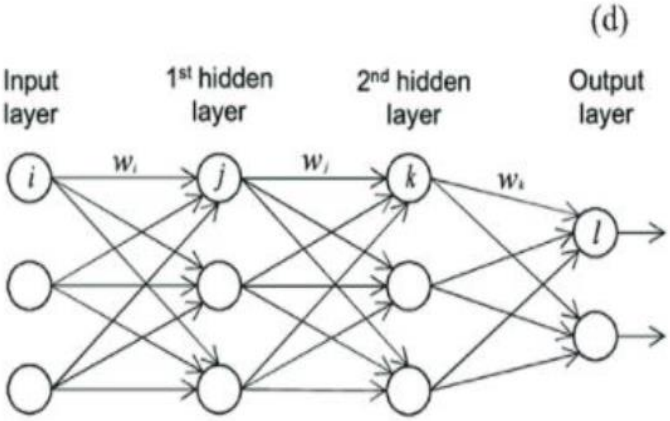
- Regularization changes the game. Instead of just minimizing Error, the model now has to minimize:
- Total Loss}= {Error} + {Penalty}
- **Error:** Being wrong on the data.
- **Penalty:** Having complex/large weights
- If the model tries to use a massive weight to fit a noisy data point, the **Penalty** goes up.[2] The model realizes, "It's too expensive to memorize this," and chooses a simpler answer

Neural Network





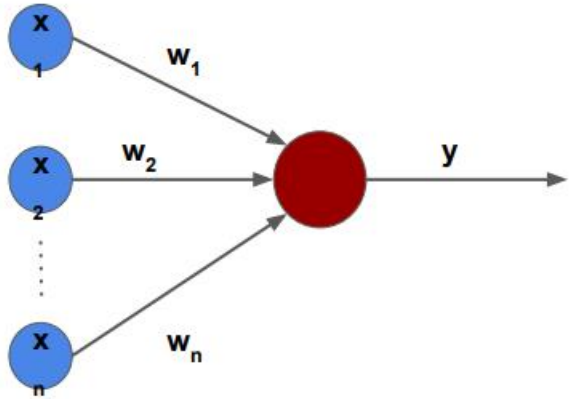




The Perceptron Model

- Motivated by the biological neuron.
- A perceptron is a computing element where inputs are associated with the weights and the cell having a threshold value.

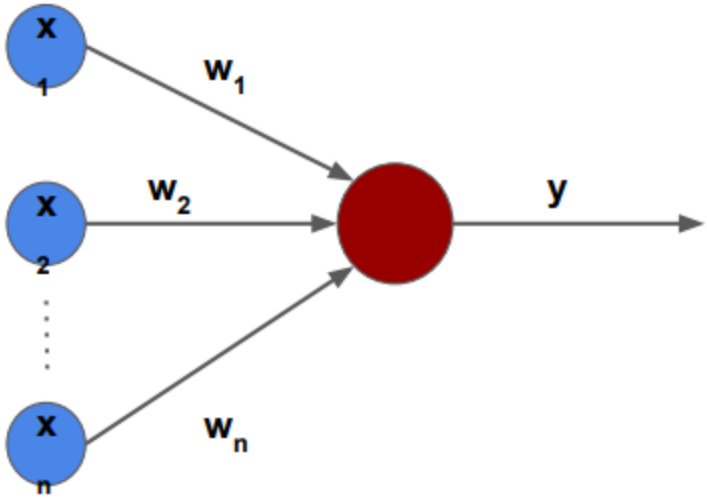
$$y = \begin{cases} 1, & \text{if } \sum w_i x_i > \text{threshold} \\ 0 & \text{otherwise} \end{cases}$$



The Perceptron Model

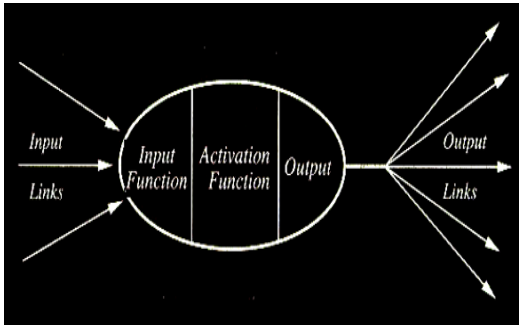
- Rewrite $\Sigma\, w_i x_i$ as $w.x$
- Replace threshold = -b
- **b:** Bias, a prior inclination towards some decision.

$$y = \begin{cases} 1, & \text{if } w.x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

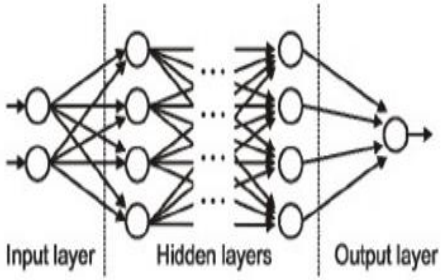


Key Concepts

➢ **Neurons:** Neurons are the fundamental building blocks of a neural network. Neurons are used for receiving input, processing input and generating output. Artificial neural networks are made up of neurons. Depending on the layer in which neurons are present they perform different functionalities.

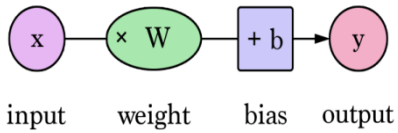


➢ **Layers(Input/hidden/output):** The neurons are organized in the form of layers in an ANN. All layers in a neural network can be segregated as either a input layer, hidden layer or an output layer. The input layer receives the input and is the first layer of the neural network. The hidden layers are the processing layers that learn the features or patterns for mapping the input to the output layer. The output layer is the final layer that generates the output.



Key Concepts

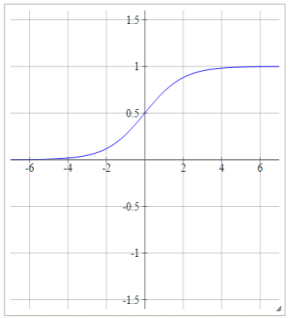
➢ **Weights and biases:** The neurons in a neural network are connected and each connection has an associated weight assigned to it. We initialize the weights randomly and these weights are updated during the model training process.



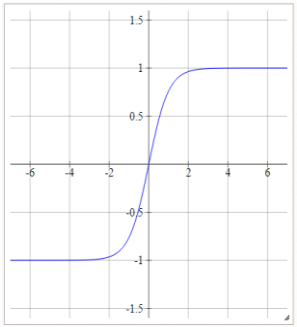
Biases are analogous to the intercept in a regression model.

➢**Activation functions:** Activation functions introduce non-linearity into neural networks. With these activation functions, neural networks will be very similar to that of a linear model. Commonly used activation functions include sigmoid, tanh, ReLu and softmax.

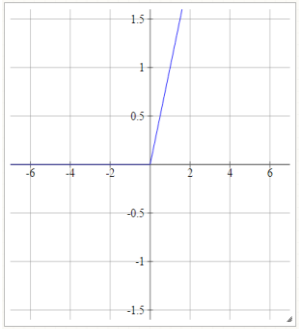
Sigmoid = 1/(1+e^(-x))



Tanh= (e^(x) – e^(-x))/((e^(x) + e^(-x)))



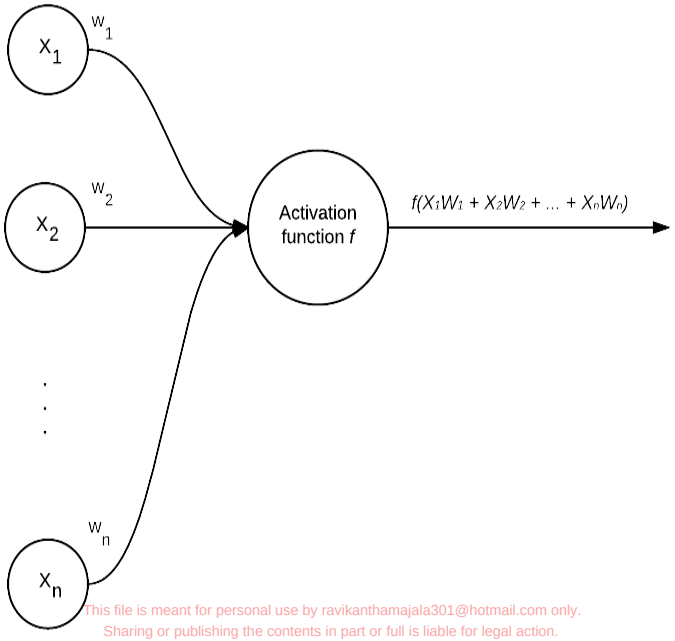
ReLu=max(0,x)



NEURAL NETWORKS

- Neural networks are the building blocks of deep learning systems.
- Each of us contains a real-life biological neural network that is connected to our nervous system — this network is made up of a large number of interconnected neurons (nerve cells).
- The word “neural” is the adjective form of “neuron”, and “network” denotes a graph-like structure; therefore, an Artificial Neural Network is a computation system that attempts to mimic the neural connections in our nervous system.

- Each node performs a simple computation. Each connection then carries a “signal” (i.e., the output of the computation) from one node to another, labeled by a “weight” indicating the extent to which the signal is amplified or diminished.
- Some connections have large, positive weights that amplify the signal, indicating that the signal is very important in making a classification. Others have negative weights, diminishing the strength of the signal, thus specifying that the output of the node is less important in the final class



pixel image

imread

Blue

Green

Red

255	134	93	22
255	134	202	22
255	231	42	22
123	94	83	2
34	44	187	92
34	76	232	124
67	83	194	202

reshaped image vector

$$\begin{pmatrix} 255 \\ 231 \\ 42 \\ 22 \\ 123 \\ 94 \\ \vdots \\ \vdots \\ 92 \\ 142 \end{pmatrix}$$

- The values X1, X2,…Xn are the inputs to our NN. These inputs could be raw pixel intensities or the entries in a feature vector.
- Each X is connected to a neuron via a weight vector, W, consisting of w1, w2,…wn. This means that for each input x, we also have an associated weight w.

$$f\left(w_1 x_1 + w_2 x_2 + \cdots + w_n x_n\right)$$

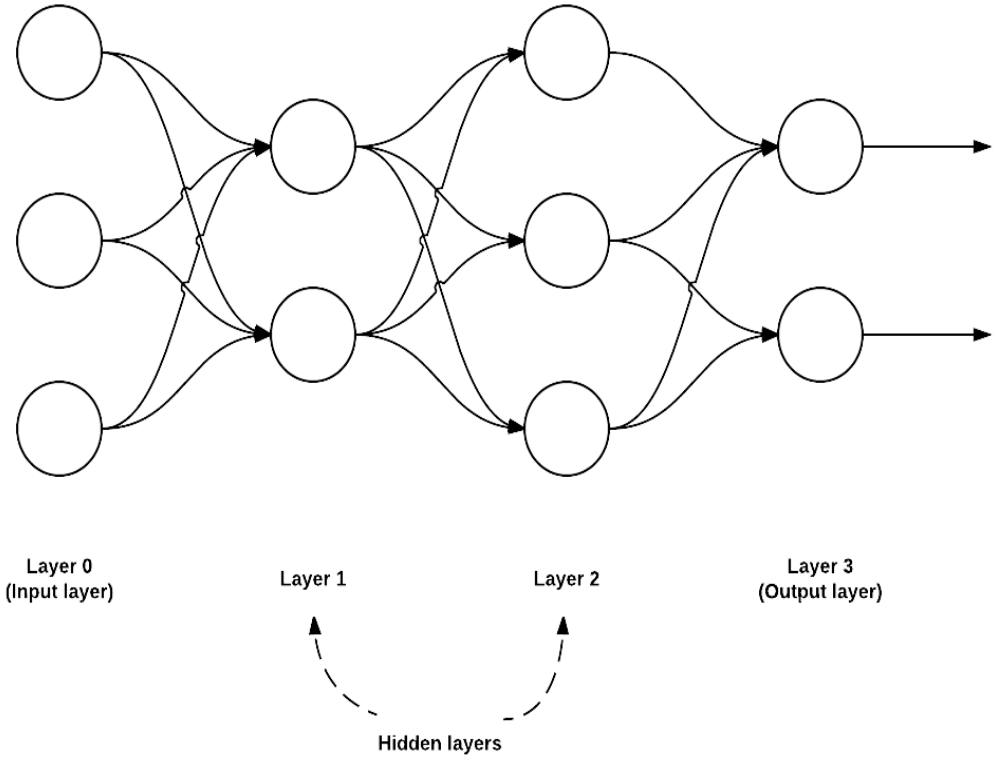
$$net = \sum_{i=1}^{n} w_i x_i$$

$$f\left(\sum_{i=1}^{n} w_i x_i\right)$$

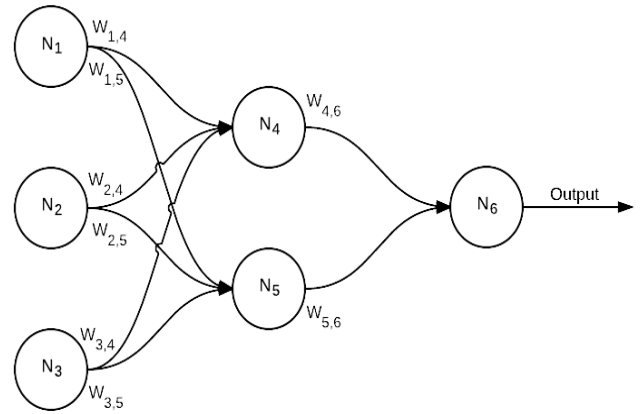
$$f(net) = \begin{cases} 1 \text{ if } net > 0 \\ 0 \text{ otherwise} \end{cases}$$

NETWORK ARCHITECTURE

- While there are many, many different NN architectures, the most common architecture is the feedforward network.



- In this type of architecture, a connection between nodes is only allowed from nodes in layer i to nodes in layer i + 1 (hence the term feedforward; there are no backwards or inter-layer connections allowed).
- **Layer 0** contains 3 inputs, our values. These could be pixel intensities or entries from a feature vector.
- **Layers 1 and 2** are **hidden layers** containing 2 and 3 nodes, respectively.
- **Layer 3** is the **output layer** or the **visible layer**



The computation hidden layer neurons are N4 and N5.

$$N_4 = f(w_{1,4} \times N_1 + w_{2,4} \times N_2 + w_{3,4} \times N_3)$$

$$N_5 = f(w_{1,5} \times N_1 + w_{2,5} \times N_2 + w_{3,5} \times N_3)$$

computation output layer neuron is O.

$$o = N_6 = f(w_{4,6} \times N_4 + w_{5,6} \times N_5)$$

Loss function computation.

$$err = y - o$$

image2vector
standardize

x_0

x_1

x_{12286}

x_{12287}

Linear Relu

$W^{[1]}x + b^{[1]}$ | RELU

$a_0^{[1]}$

$a_1^{[1]}$

$a_{n^{[1]}-2}^{[1]}$

$a_{n^{[1]}-1}^{[1]}$

Linear Sigmoid

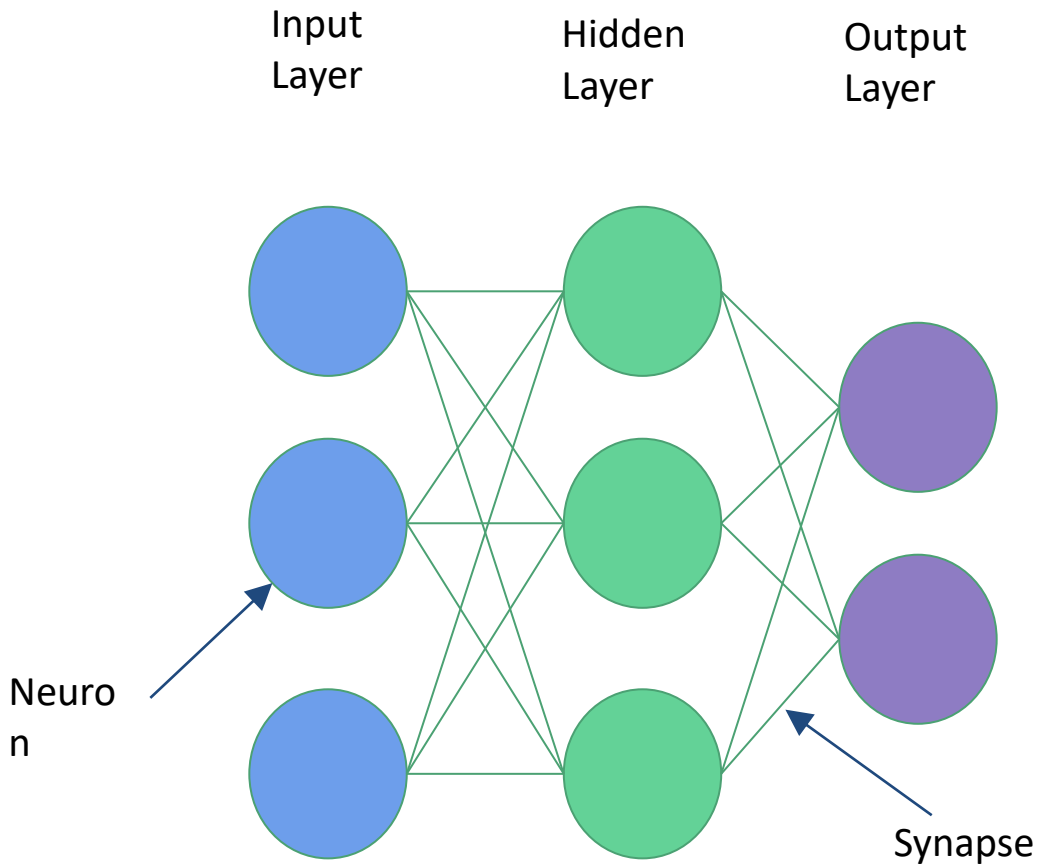
$W^{[2]}a^{[1]} + b^{[2]}$ | σ → 0.73

0.73 > 0.5

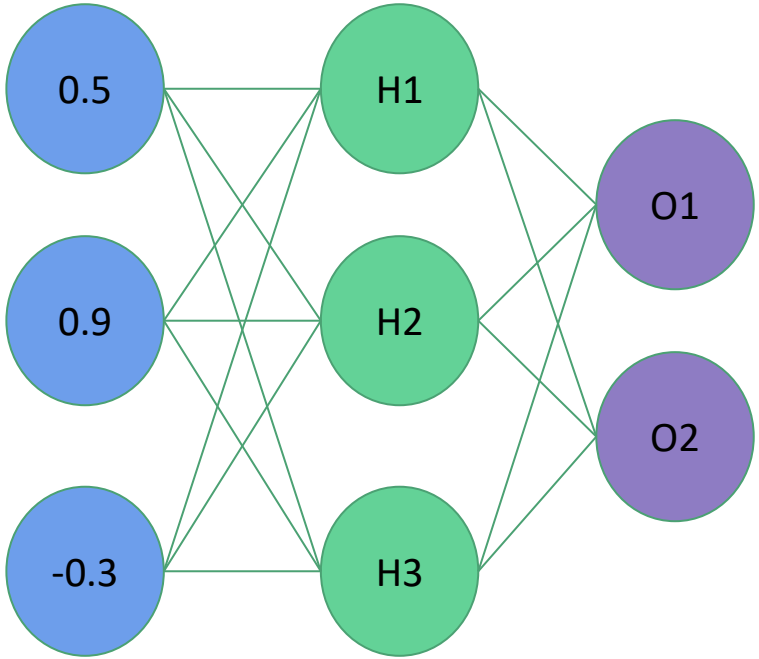
probability cat more than probability non-cat

"it's a cat"

Neural Network Architecture



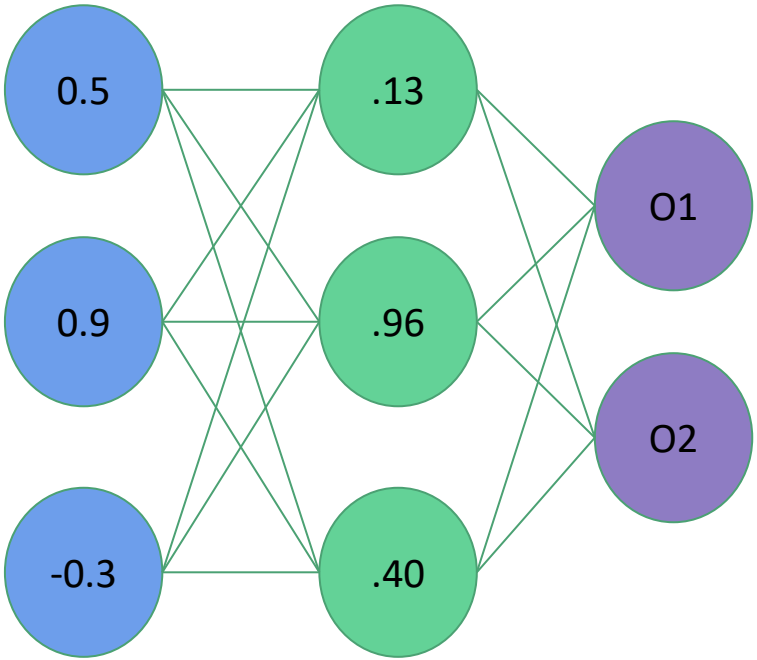
Inference



H1 Weights = (1.0, -2.0, 2.0)
H2 Weights = (2.0, 1.0, -4.0)
H3 Weights = (1.0, -1.0, 0.0)

O1 Weights = (-3.0, 1.0, -3.0)
O2 Weights = (0.0, 1.0, 2.0)

Inference

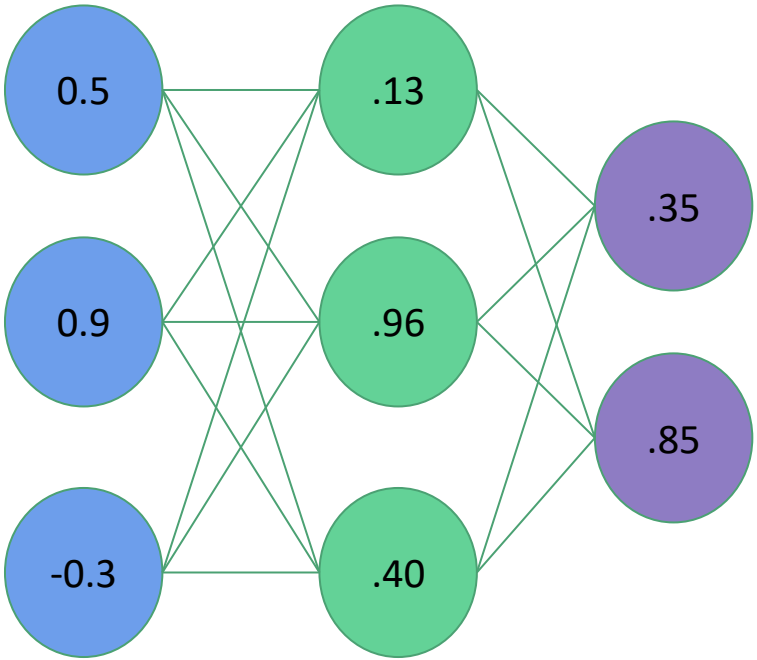


H1 Weights = (1.0, -2.0, 2.0)
H2 Weights = (2.0, 1.0, -4.0)
H3 Weights = (1.0, -1.0, 0.0)

O1 Weights = (-3.0, 1.0, -3.0)
O2 Weights = (0.0, 1.0, 2.0)

H1 = S(0.5 * 1.0 + 0.9 * -2.0 + -0.3 * 2.0) = S(-1.9) = .13
H2 = S(0.5 * 2.0 + 0.9 * 1.0 + -0.3 * -4.0) = S(3.1) = .96
H3 = S(0.5 * 1.0 + 0.9 * -1.0 + -0.3 * 0.0) = S(.40

Inference



H1 Weights = (1.0, -2.0, 2.0)
H2 Weights = (2.0, 1.0, -4.0)
H3 Weights = (1.0, -1.0, 0.0)

O1 Weights = (-3.0, 1.0, -3.0)
O2 Weights = (0.0, 1.0, 2.0)

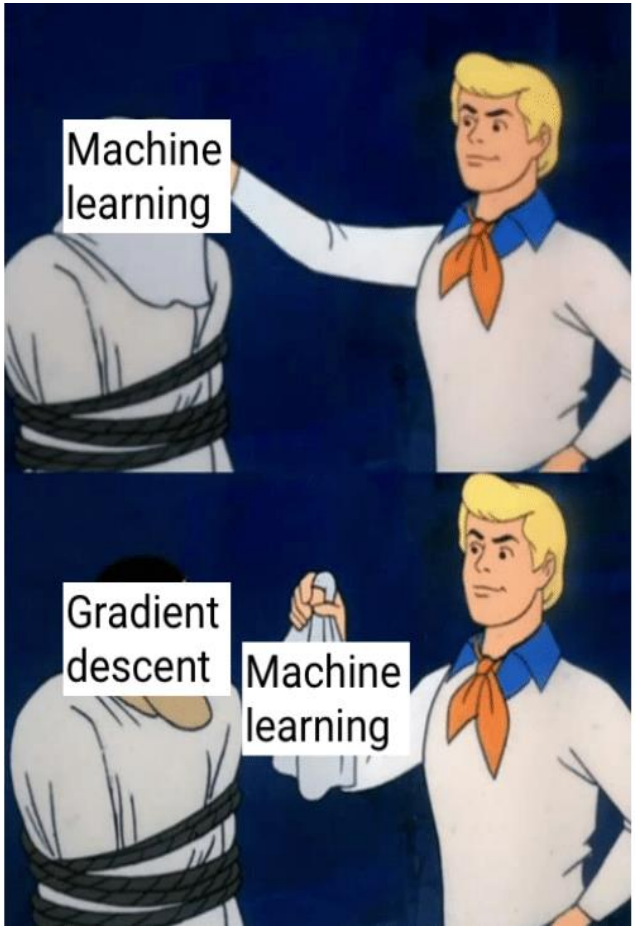
O1 = S(.13 * -3.0 + .96 * 1.0 + .40 * -3.0) = S(-.63) = .35

O2 = S(.13 * 0.0 + .96 * 1.0 + .40 * 2.0) = S(1.76) = .85

Need Optimizers

$$L = -\sum_{i=1}^{2} t_i \log(p_i)$$
$$= -\left[t \log(p) + (1-t) \log(1-p)\right]$$

where t_i is the truth value taking a value 0 or 1 and p_i is the Softmax probability for the i^{th} class.

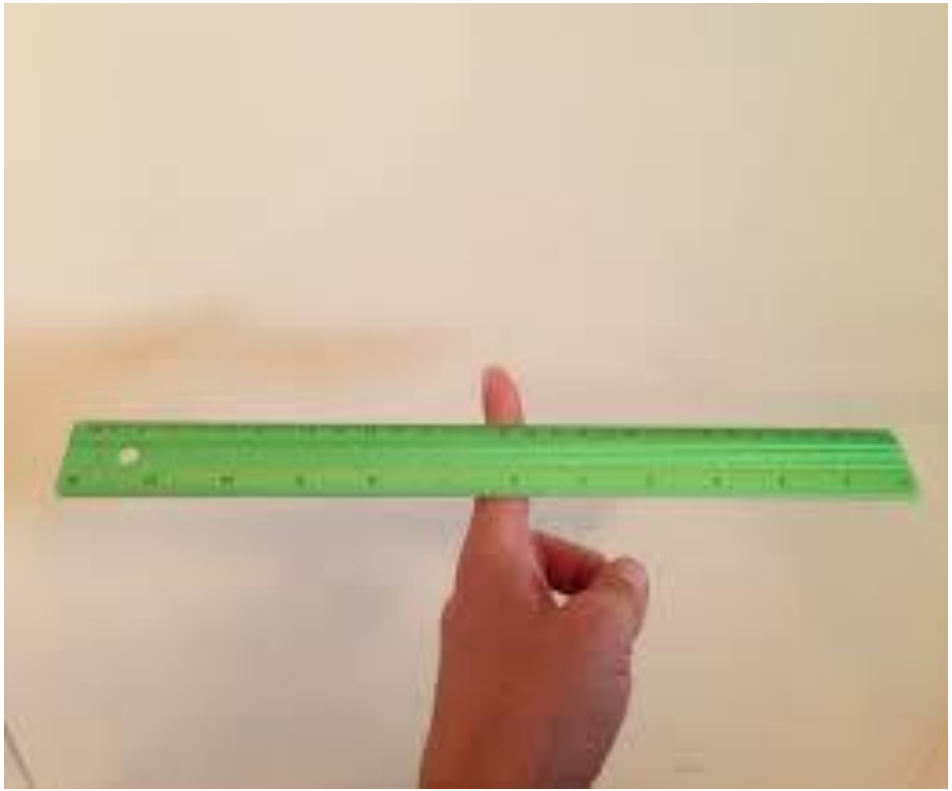


Weight Calculation

w = w - (α x (Grad(loss)))

Gradient and Gradient Free Optimizer

Gradient Free Optimizers

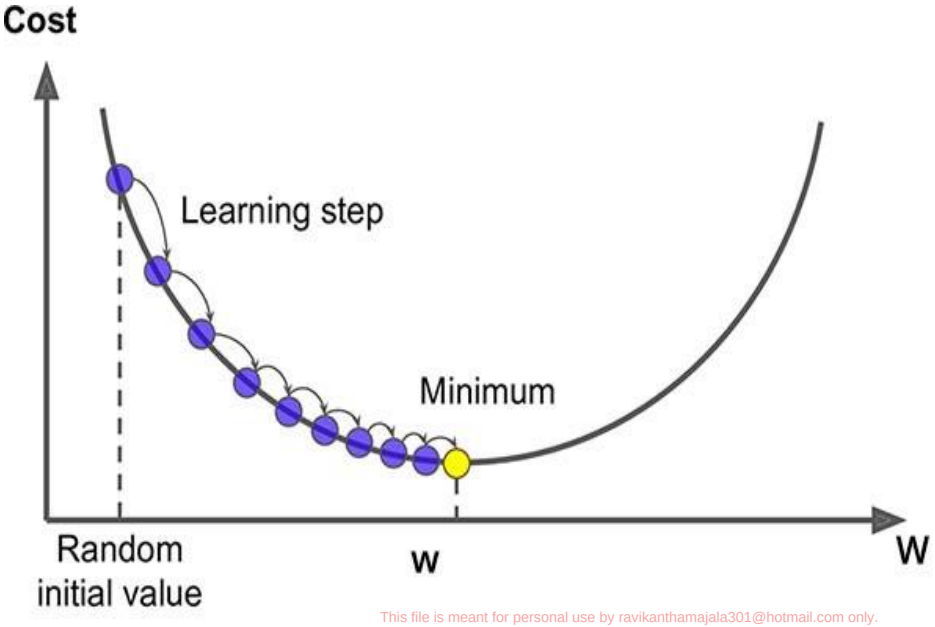


Gradient based optimization

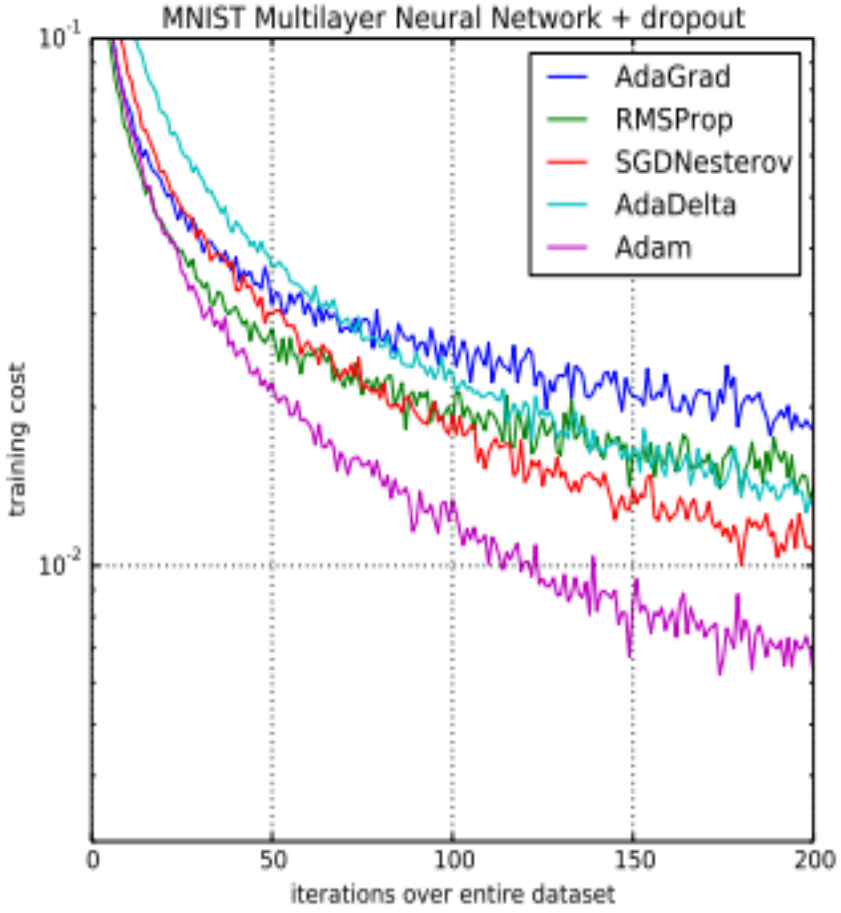
Vanilla, SGD & Mini Batch Descendants

1. **Search direction.**
2. **Step size**
3. **Convergence check.**

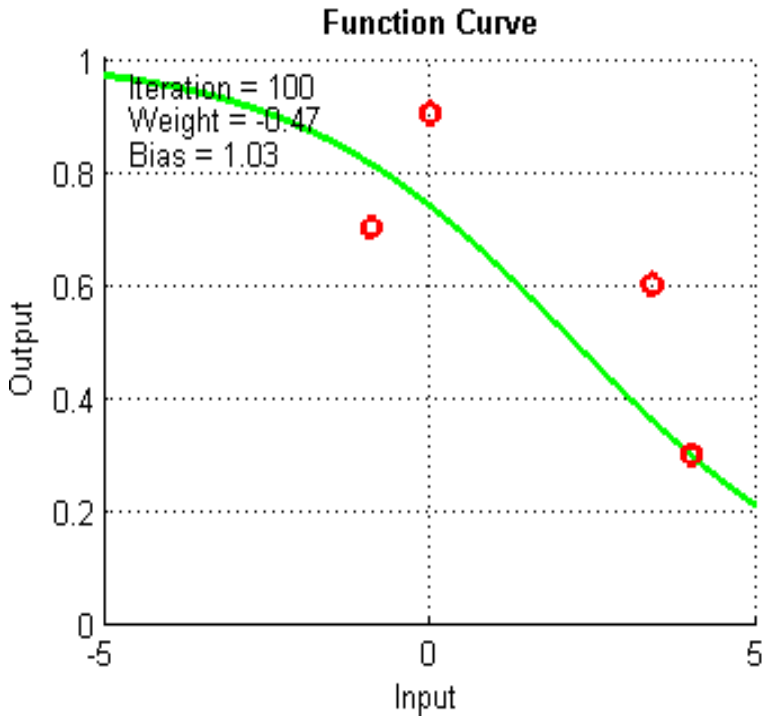
Big steps when away, small steps when closer.

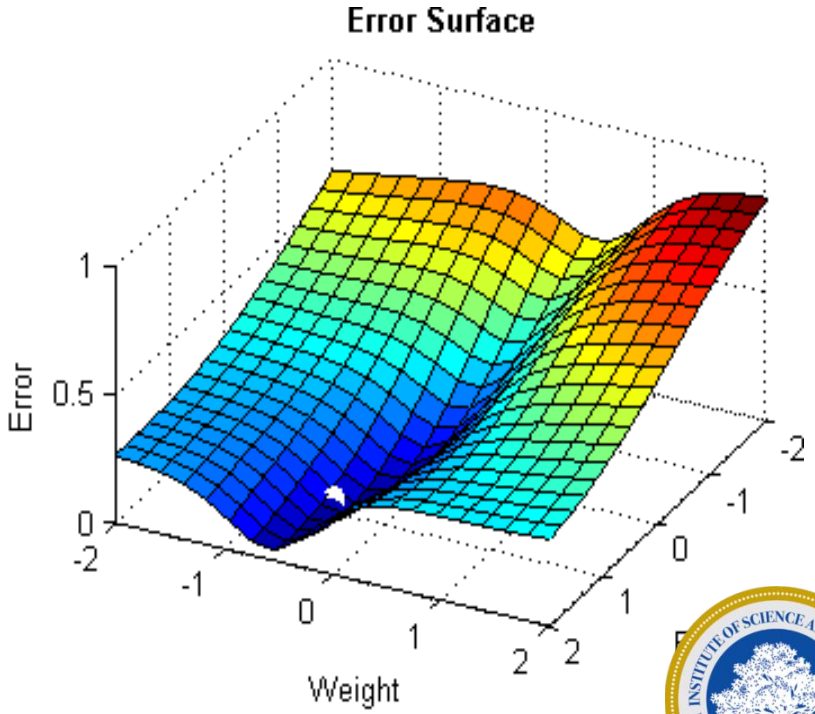


Optimizers



GD





Semester 4

End of Slide

THANK YOU